DriverJinn — Downstream Analysis & Poster Plots

Setup

Training Curves

```
KEY = "val_ndcg@50"
fig, ax = plt.subplots(figsize=(8, 6))
for fold, history in sorted(histories.items()):
    if KEY not in history or len(history[KEY]) == 0:
        continue
    y = np.array(history[KEY])
    ax.plot(np.arange(1, len(y) + 1), y,
            label=f"Fold {fold}", color=FOLD_COLORS[fold - 1], alpha=0.85, linewidth=1.5)
ax.set_xlabel("Epoch", fontsize=12)
ax.set_ylabel("NDCG@50", fontsize=12)
ax.set_title("Validation NDCG@50 Across Folds", fontsize=14, fontweight="bold")
ax.legend(loc="best", fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

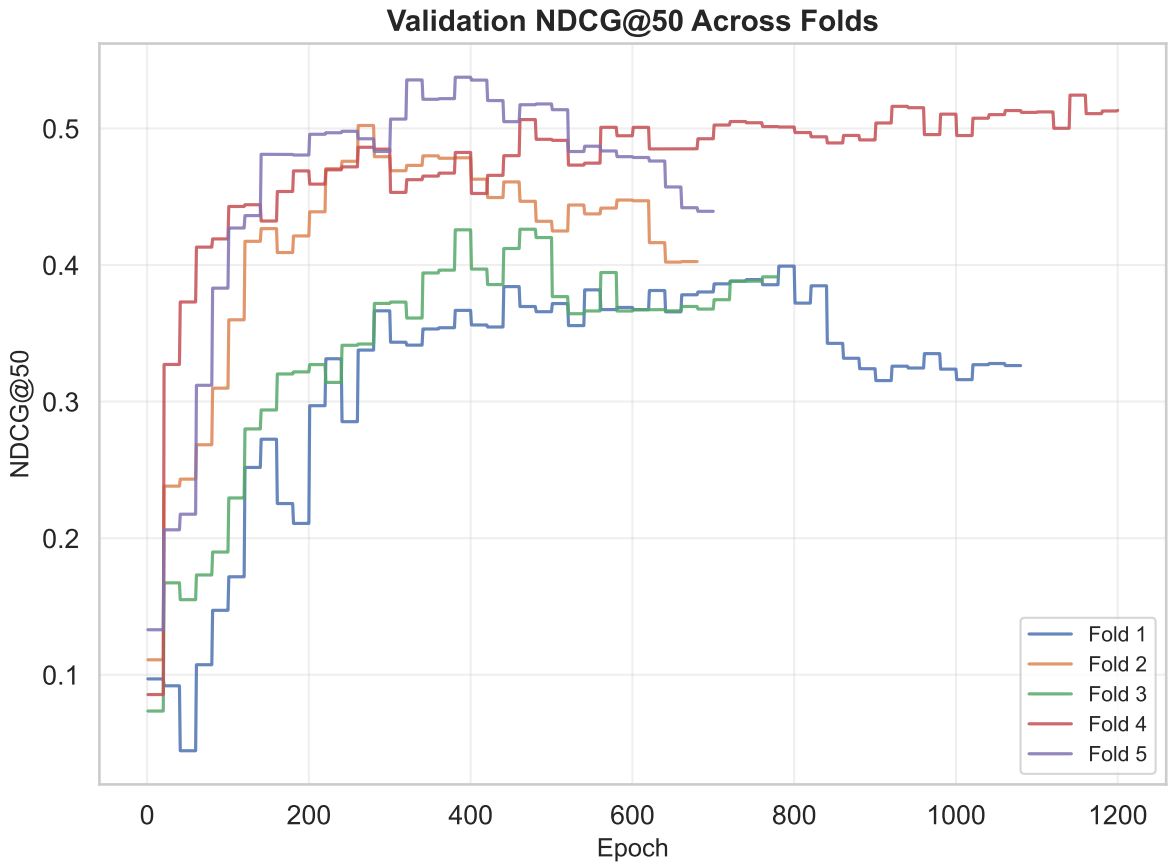


Figure 1: Validation NDCG@50 across folds.

```
KEY = "val_auroc"
fig, ax = plt.subplots(figsize=(8, 6))
for fold, history in sorted(histories.items()):
    if KEY not in history or len(history[KEY]) == 0:
        continue
    y = np.array(history[KEY])
    ax.plot(np.arange(1, len(y) + 1), y,
            label=f"Fold {fold}", color=FOLD_COLORS[fold - 1], alpha=0.85, linewidth=1.5)
ax.set_xlabel("Epoch", fontsize=12)
ax.set_ylabel("AUROC", fontsize=12)
ax.set_title("Validation AUROC Across Folds", fontsize=14, fontweight="bold")
ax.legend(loc="lower right", fontsize=10)
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
ax.grid(True, alpha=0.3)
plt.tight_layout()
```

```
plt.show()
```

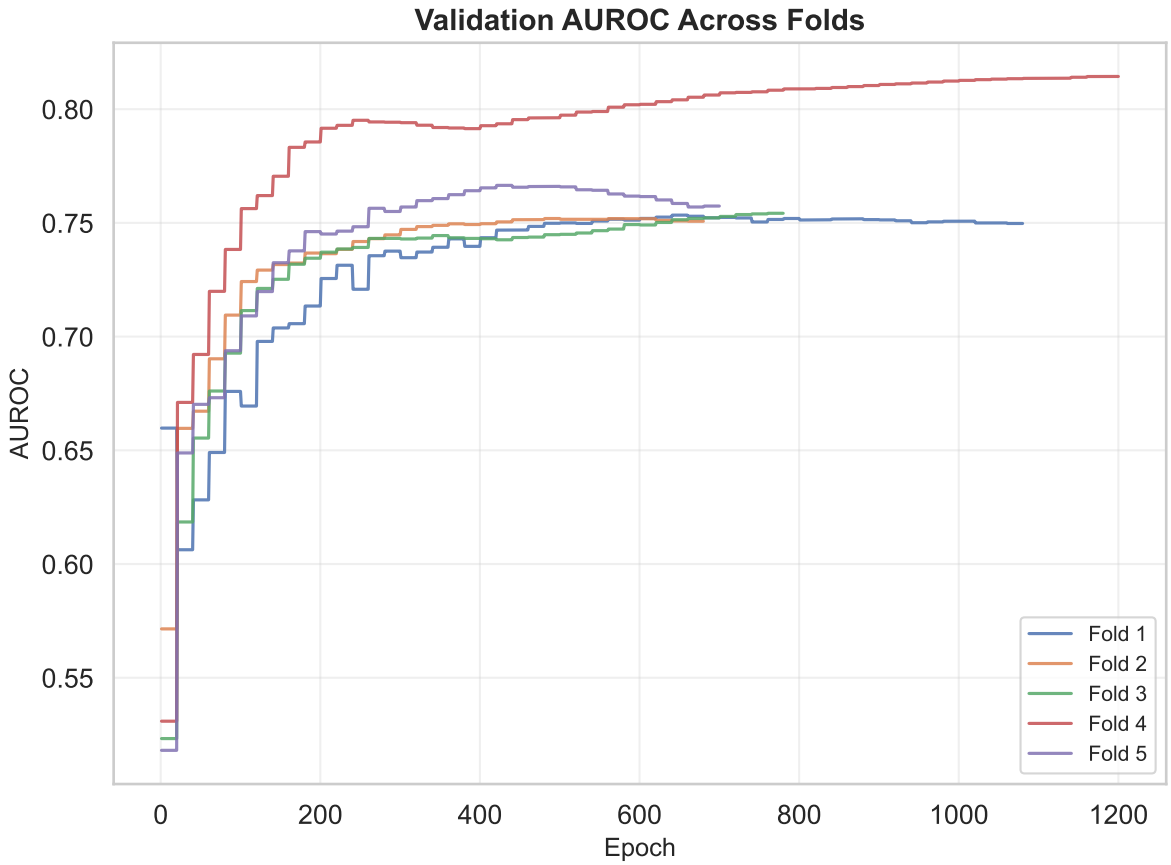


Figure 2: Validation AUROC across folds.

```
rows = []
for fold, history in sorted(histories.items()):
    ndcg  = history.get("val_ndcg@50", [])
    auroc = history.get("val_auroc", [])
    rows.append({
        "Fold":         fold,
        "Peak NDCG@50": round(max(ndcg),  4) if ndcg  else None,
        "Peak AUROC":   round(max(auroc), 4) if auroc else None,
        "Epochs":       len(ndcg),
    })
df_summary = pd.DataFrame(rows)
mean_row = {"Fold": "Mean",
```

```
            "Peak NDCG@50": round(df_summary["Peak NDCG@50"].mean(), 4),
            "Peak AUROC":   round(df_summary["Peak AUROC"].mean(),   4),
            "Epochs":       "-"}
df_summary = pd.concat([df_summary, pd.DataFrame([mean_row])], ignore_index=True)
df_summary.style.set_caption("Per-fold peak validation metrics")
```

Table 1: Per-fold peak validation metrics

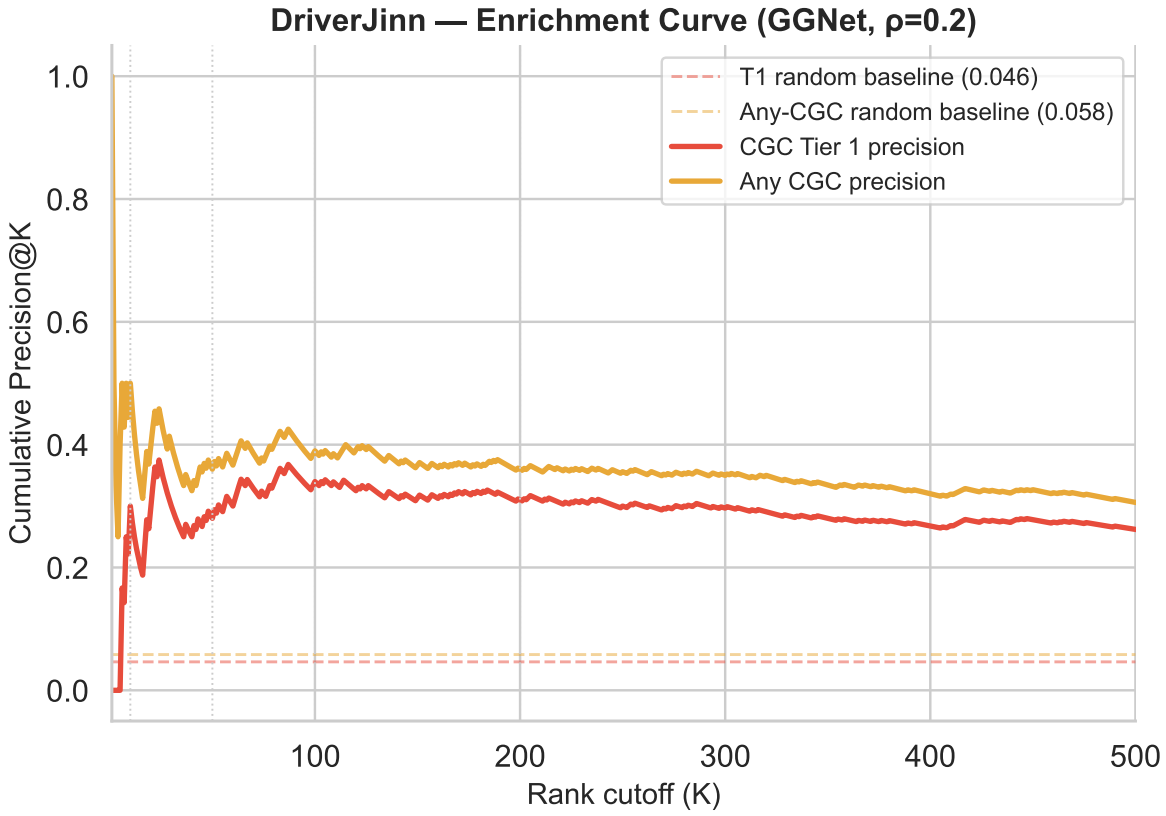
	Fold	Peak NDCG@50	Peak AUROC	Epochs
0	1	0.399200	0.753400	1080
1	2	0.502200	0.751900	680
2	3	0.426200	0.754200	780
3	4	0.524400	0.814400	1200
4	5	0.537500	0.766500	700
5	Mean	0.477900	0.768100	—

Enrichment Curve (Cumulative CGC Precision)

How quickly DriverJinn recovers known cancer driver genes as you move down the ranked list.

```
fig, ax = plt.subplots(figsize=(7, 5))
ks = np.arange(1, len(genes) + 1)
cumulative_t1  = np.cumsum(genes["cgc_tier"] == "T1")  / ks
cumulative_any = np.cumsum(genes["cgc_tier"] != "non-CGC") / ks
baseline_t1  = (genes["cgc_tier"] == "T1").sum()  / len(genes)
baseline_any = (genes["cgc_tier"] != "non-CGC").sum() / len(genes)

ax.axhline(baseline_t1,  color=COLORS["T1"], linestyle="--", linewidth=1.2,
           alpha=0.5, label=f"T1 random baseline ({baseline_t1:.3f})")
ax.axhline(baseline_any, color=COLORS["T2"], linestyle="--", linewidth=1.2,
           alpha=0.5, label=f"Any-CGC random baseline ({baseline_any:.3f})")
ax.plot(ks, cumulative_t1,  color=COLORS["T1"], linewidth=2.2, label="CGC Tier 1 precision")
ax.plot(ks, cumulative_any, color=COLORS["T2"], linewidth=2.2, label="Any CGC precision")
for k in [10, 50, 100, 200]:
    if k <= len(genes):
        ax.axvline(k, color="#CCCCCC", linewidth=0.8, linestyle=":")
ax.set_xlim(1, 500)
```

```
ax.set_xlabel("Rank cutoff (K)", fontsize=12)
ax.set_ylabel("Cumulative Precision@K", fontsize=12)
ax.set_title("DriverJinn - Enrichment Curve (GGNet,  =0.2)", fontsize=13, fontweight="bold")
ax.legend(fontsize=10)
sns.despine()
plt.tight_layout()
plt.savefig(PLOTS_DIR / "poster_enrichment_curve.png", dpi=300, bbox_inches="tight")
plt.show()
```

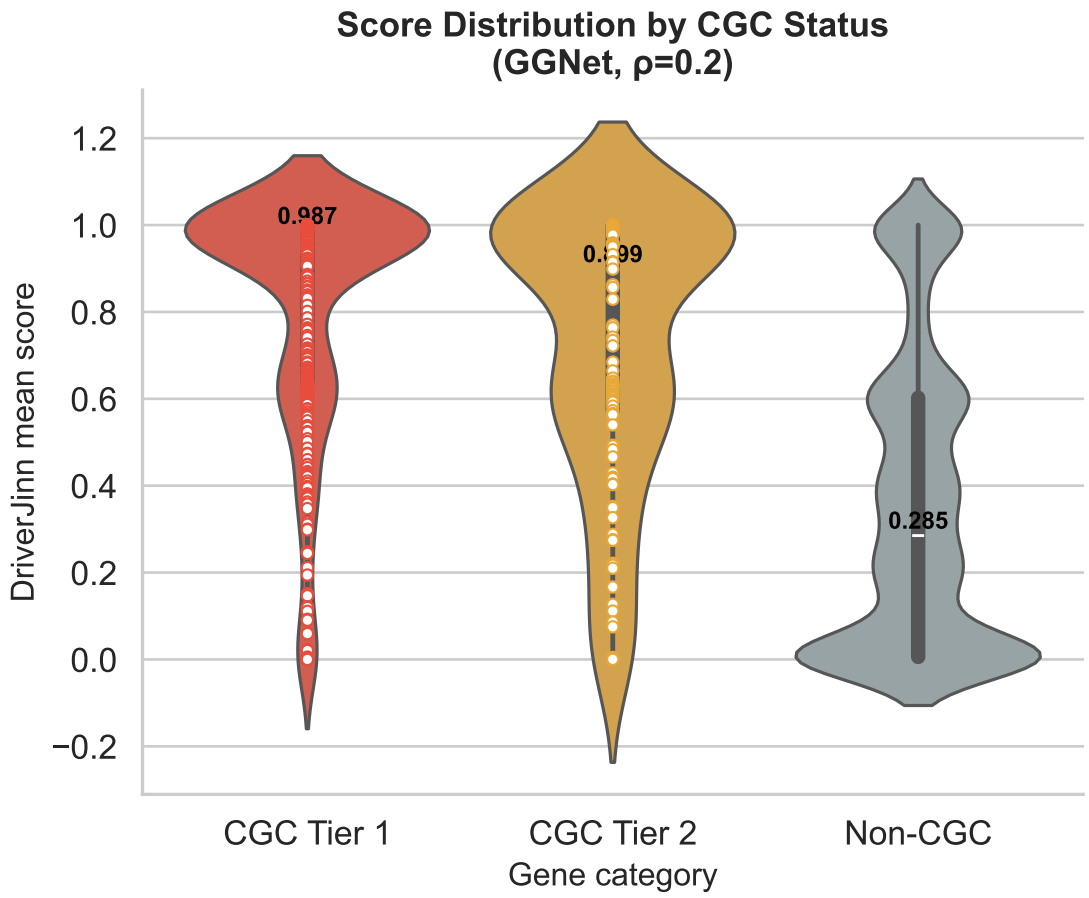


Score Distribution by CGC Tier

Separation of DriverJinn scores across CGC Tier 1, Tier 2, and non-CGC genes.

```
fig, ax = plt.subplots(figsize=(6, 5))
order   = ["T1", "T2", "non-CGC"]
palette = {k: COLORS[k] for k in order}
sns.violinplot(data=genes, x="cgc_tier", y="mean_score",
               order=order, palette=palette, inner="box", linewidth=1.2, ax=ax)
for tier, color in [("T1", COLORS["T1"]), ("T2", COLORS["T2"])]:
    sub = genes[genes["cgc_tier"] == tier]
    ax.scatter([order.index(tier)] * len(sub), sub["mean_score"],
               color="white", edgecolors=color, s=18, zorder=5, linewidth=0.8)
for i, tier in enumerate(order):
    med = genes.loc[genes["cgc_tier"] == tier, "mean_score"].median()
    ax.text(i, med + 0.005, f"{med:.3f}", ha="center", va="bottom",
            fontsize=9, fontweight="bold", color="black")
ax.set_xlabel("Gene category", fontsize=12)
ax.set_ylabel("DriverJinn mean score", fontsize=12)
ax.set_title("Score Distribution by CGC Status\n(GGNet,  =0.2)", fontsize=13, fontweight="bol
ax.set_xticklabels(["CGC Tier 1", "CGC Tier 2", "Non-CGC"])
sns.despine()
plt.tight_layout()
plt.savefig(PLOTS_DIR / "poster_score_violin.png", dpi=300, bbox_inches="tight")
plt.show()
```

```
C:\Users\siddh\AppData\Local\Temp\ipykernel_81720\746990387.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assig

  sns.violinplot(data=genes, x="cgc_tier", y="mean_score",
C:\Users\siddh\AppData\Local\Temp\ipykernel_81720\746990387.py:17: UserWarning: set_ticklabel
  ax.set_xticklabels(["CGC Tier 1", "CGC Tier 2", "Non-CGC"])
```

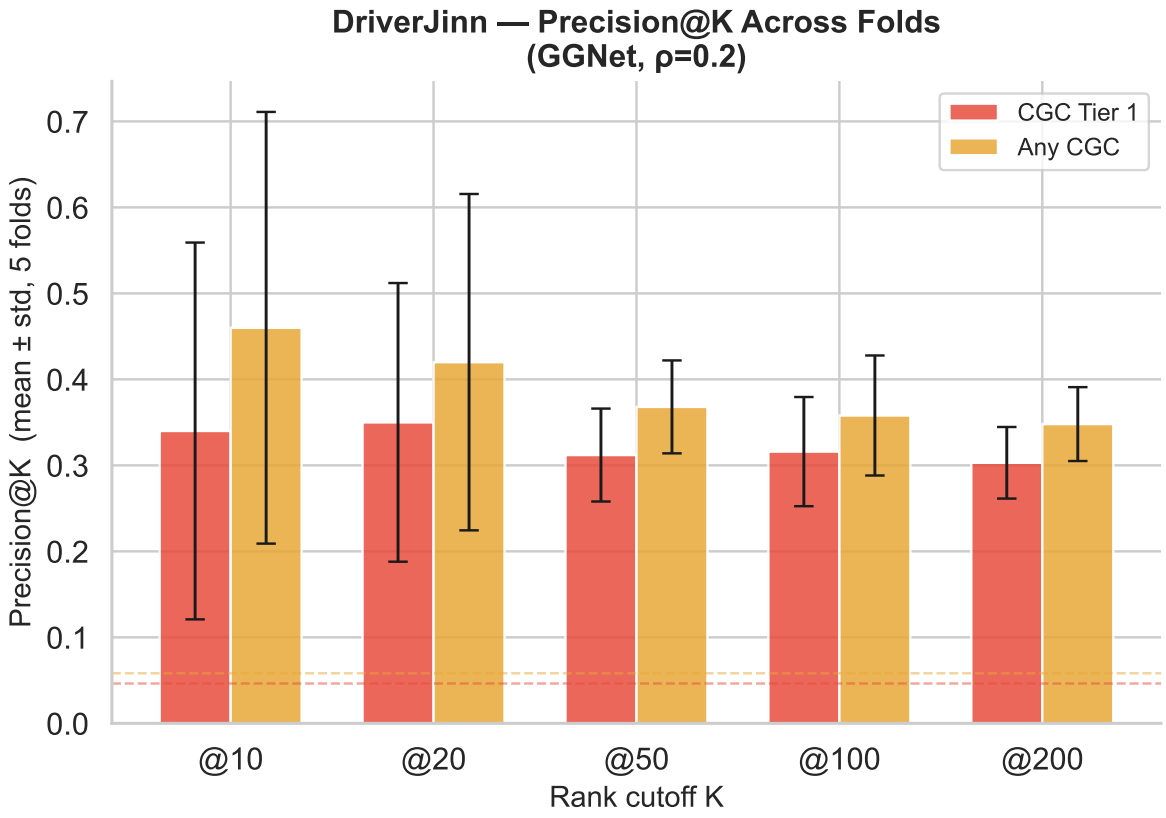


Precision@K Across Folds

Per-fold and mean Precision at multiple rank cutoffs, with error bars.

```
ks   = [10, 20, 50, 100, 200]
rows = []
for fold, df in fold_dfs.items():
    df = df.sort_values("rank").reset_index(drop=True)
    df["cgc_t1"]  = df["gene_name"].isin(cgc_t1)
    df["cgc_any"] = df["gene_name"].isin(cgc_t1 | cgc_t2)
    for k in ks:
        top_k = df.head(k)
        rows.append({"fold": fold, "K": k,
```

```
                    "Precision@K (T1)":      top_k["cgc_t1"].mean(),
                    "Precision@K (Any CGC)": top_k["cgc_any"].mean()})
prec_df = pd.DataFrame(rows)
summary = (prec_df.groupby("K")[["Precision@K (T1)", "Precision@K (Any CGC)"]]
           .agg(["mean", "std"]).reset_index())
summary.columns = ["K", "t1_mean", "t1_std", "any_mean", "any_std"]

fig, ax = plt.subplots(figsize=(7, 5))
width, xs = 0.35, np.arange(len(ks))
ax.bar(xs - width/2, summary["t1_mean"],  width, yerr=summary["t1_std"],
       color=COLORS["T1"], alpha=0.85, capsize=4, label="CGC Tier 1",
       error_kw={"linewidth": 1.2})
ax.bar(xs + width/2, summary["any_mean"], width, yerr=summary["any_std"],
       color=COLORS["T2"], alpha=0.85, capsize=4, label="Any CGC",
       error_kw={"linewidth": 1.2})
baseline_t1_v  = len(cgc_t1 & set(genes["gene_name"])) / len(genes)
baseline_any_v = len((cgc_t1 | cgc_t2) & set(genes["gene_name"])) / len(genes)
ax.axhline(baseline_t1_v,  color=COLORS["T1"], linestyle="--", linewidth=1, alpha=0.5)
ax.axhline(baseline_any_v, color=COLORS["T2"], linestyle="--", linewidth=1, alpha=0.5)
ax.set_xticks(xs)
ax.set_xticklabels([f"@{k}" for k in ks])
ax.set_xlabel("Rank cutoff K", fontsize=12)
ax.set_ylabel("Precision@K  (mean ± std, 5 folds)", fontsize=12)
ax.set_title("DriverJinn - Precision@K Across Folds\n(GGNet,  =0.2)", fontsize=13, fontweight
ax.legend(fontsize=10)
sns.despine()
plt.tight_layout()
plt.savefig(PLOTS_DIR / "poster_precision_at_k.png", dpi=300, bbox_inches="tight")
plt.show()
```

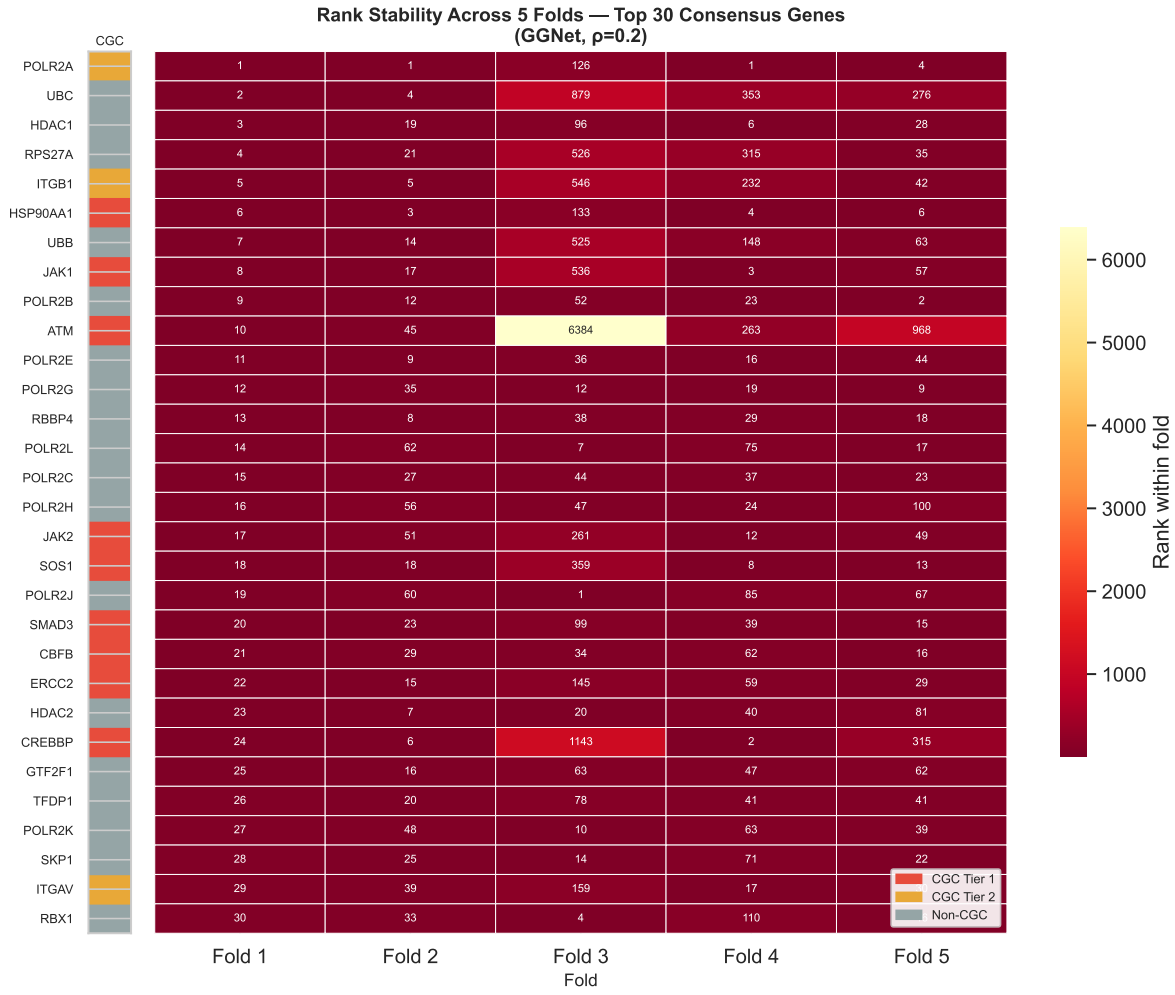


Rank Stability Heatmap

For the consensus top-30 genes, how consistent is their rank across all 5 folds?

```
TOP_N     = 30
top_genes = genes.head(TOP_N)["gene_name"].tolist()
rank_matrix = pd.DataFrame(index=top_genes)
for fold, df in fold_dfs.items():
    df = df.sort_values("rank")
    name_to_rank = dict(zip(df["gene_name"], df["rank"]))
    rank_matrix[f"Fold {fold}"] = [name_to_rank.get(g, np.nan) for g in top_genes]

cgc_array = np.array([[
    0 if g in cgc_t1 else 1 if g in cgc_t2 else 2
    for g in top_genes
```

```
]]).T

fig, axes = plt.subplots(1, 2, figsize=(11, 9),
                         gridspec_kw={"width_ratios": [0.04, 1]})
axes[0].imshow(cgc_array, aspect="auto",
               cmap=plt.cm.colors.ListedColormap([COLORS["T1"], COLORS["T2"], COLORS["non-CGC
axes[0].set_xticks([])
axes[0].set_yticks(range(TOP_N))
axes[0].set_yticklabels(top_genes, fontsize=8)
axes[0].set_title("CGC", fontsize=8, pad=4)
sns.heatmap(rank_matrix, ax=axes[1], cmap="YlOrRd_r", annot=True, fmt=".0f",
            annot_kws={"size": 7}, linewidths=0.3,
            cbar_kws={"label": "Rank within fold", "shrink": 0.6})
axes[1].set_yticks([])
axes[1].set_xlabel("Fold", fontsize=11)
axes[1].set_title("Rank Stability Across 5 Folds - Top 30 Consensus Genes\n(GGNet,  =0.2)",
                  fontsize=12, fontweight="bold")
axes[1].legend(handles=[
    mpatches.Patch(color=COLORS["T1"],      label="CGC Tier 1"),
    mpatches.Patch(color=COLORS["T2"],      label="CGC Tier 2"),
    mpatches.Patch(color=COLORS["non-CGC"], label="Non-CGC"),
], loc="lower right", fontsize=8, framealpha=0.9)
plt.tight_layout()
plt.savefig(PLOTS_DIR / "poster_rank_stability.png", dpi=300, bbox_inches="tight")
plt.show()
```

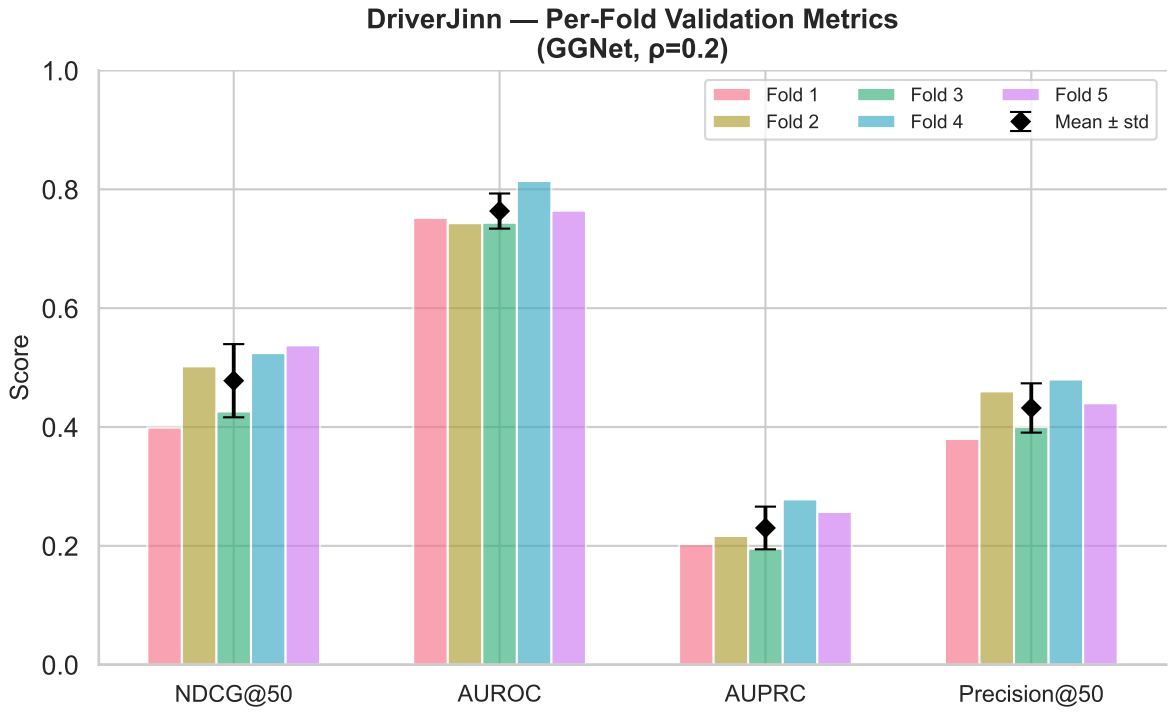


Per-Fold Metrics Summary

Overview of NDCG@50, AUROC, AUPRC, Precision@50 across all 5 folds.

```
metric_cols = ["NDCG@50", "AUROC", "AUPRC", "Precision@50"]
fold_data   = metrics_folds[["Fold"] + metric_cols].set_index("Fold")
mean_row    = metrics[metrics["Fold"] == "Mean"][metric_cols].values[0]
std_row     = metrics[metrics["Fold"] == "Std"][metric_cols].values[0]

fig, ax = plt.subplots(figsize=(8, 5))
```

```
fold_palette = sns.color_palette("husl", 5)
xs    = np.arange(len(metric_cols))
width = 0.13
for i, (fold, row) in enumerate(fold_data.iterrows()):
    ax.bar(xs + (i - 2) * width, row[metric_cols],
           width, alpha=0.65, color=fold_palette[i], label=f"Fold {fold}")
ax.errorbar(xs, mean_row, yerr=std_row, fmt="D", color="black",
            markersize=6, linewidth=1.8, capsize=5, zorder=10, label="Mean ± std")
ax.set_xticks(xs)
ax.set_xticklabels(metric_cols, fontsize=11)
ax.set_ylabel("Score", fontsize=12)
ax.set_ylim(0, 1)
ax.set_title("DriverJinn - Per-Fold Validation Metrics\n(GGNet,  =0.2)",
             fontsize=13, fontweight="bold")
ax.legend(fontsize=9, ncol=3, loc="upper right")
sns.despine()
plt.tight_layout()
plt.savefig(PLOTS_DIR / "poster_metrics_summary.png", dpi=300, bbox_inches="tight")
plt.show()
```



Hallmark Enrichment (ORA via Fisher's Exact Test)

```
def run_ora(gene_list, universe, hallmark_sets, min_gs_size=5, p_adj_method="fdr_bh"):
    gene_set = set(gene_list) & universe
    n_uni, n_query = len(universe), len(gene_set)
    rows = []
    for hallmark, gs in hallmark_sets.items():
        gs_in_uni = gs & universe
        if len(gs_in_uni) < min_gs_size:
            continue
        overlap_genes = gene_set & gs_in_uni
        a = len(overlap_genes)
        b = n_query - a
        c = len(gs_in_uni) - a
        d = n_uni - a - b - c
        _, pval = fisher_exact([[a, b], [c, d]], alternative="greater")
        rows.append({"Hallmark": hallmark, "Overlap": a,
                     "GS Size": len(gs_in_uni), "p-value": pval,
                     "Overlap Genes": ", ".join(sorted(overlap_genes))})
    if not rows:
        return pd.DataFrame()
    df = pd.DataFrame(rows)
    _, padj, _, _ = multipletests(df["p-value"], method=p_adj_method)
    df["p.adj"] = padj
    return df.sort_values("p-value").reset_index(drop=True)

hallmark_sets = (
    hallmarks_raw.dropna(subset=["HALLMARK"])
    .query("HALLMARK != ''")
    .groupby("HALLMARK")["GENE_SYMBOL"].apply(set).to_dict()
)
universe  = set(ranked_genes["gene_name"])
top100    = set(ranked_genes.head(100)["gene_name"])
consensus = set(ranked_genes.loc[ranked_genes["consensus_significant"], "gene_name"]) \
            if "consensus_significant" in ranked_genes.columns else top100
```

Top 100 Predicted Drivers

```
ora_top100 = run_ora(top100, universe, hallmark_sets)
if ora_top100.empty or (ora_top100["p.adj"] >= 0.05).all():
```

```
    print("No significant hallmark enrichment at p.adj < 0.05 for top 100")
else:
    sig = ora_top100[ora_top100["p.adj"] < 0.05].copy()
    sig["-log10(p.adj)"] = -np.log10(sig["p.adj"])
    fig, ax = plt.subplots(figsize=(7, max(3, len(sig) * 0.45)))
    sns.barplot(data=sig, x="-log10(p.adj)", y="Hallmark", palette="Blues_r", ax=ax)
    ax.axvline(-np.log10(0.05), color="red", linestyle="--", lw=1, label="p.adj = 0.05")
    ax.set_title("Hallmark Enrichment - Top 100 Predicted Drivers")
    ax.legend()
    plt.tight_layout()
    plt.savefig('plots/cgc_hallmark_enrichment.png')
    plt.show()

    display(sig[["Hallmark", "Overlap", "GS Size", "p-value", "p.adj"]]
            .style.format({"p-value": "{:.3e}", "p.adj": "{:.3e}"}))
```

```
C:\Users\siddh\AppData\Local\Temp\ipykernel_81720\3177314145.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assig

  sns.barplot(data=sig, x="-log10(p.adj)", y="Hallmark", palette="Blues_r", ax=ax)
```

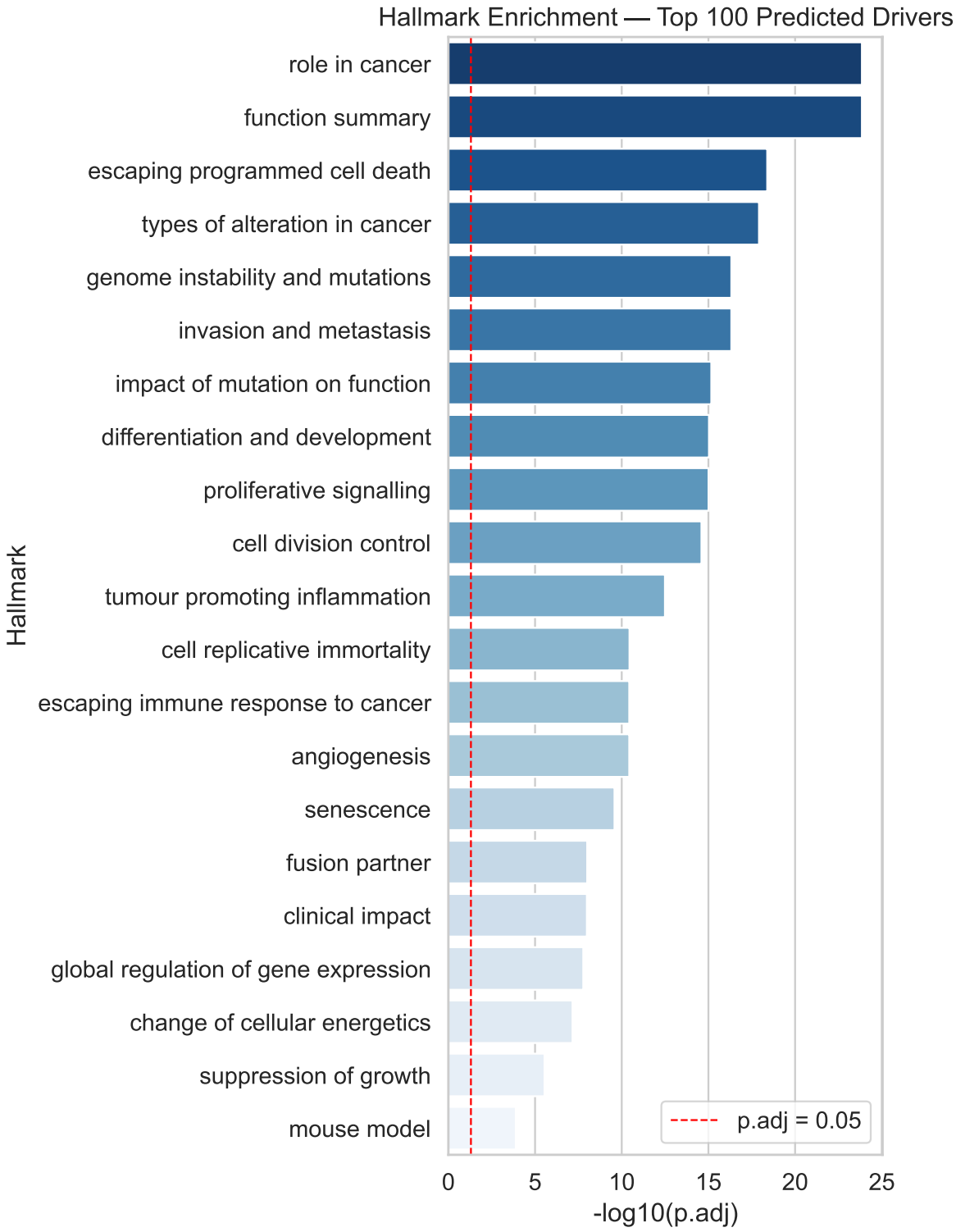

Hallmark Enrichment — Top 100 Predicted Drivers
role in cancer
function summary
escaping programmed cell death
types of alteration in cancer
genome instability and mutations
invasion and metastasis
impact of mutation on function
differentiation and development
proliferative signalling
cell division control
tumour promoting inflammation
cell replicative immortality
escaping immune response to cancer
angiogenesis
senescence
fusion partner
clinical impact
global regulation of gene expression
change of cellular energetics
suppression of growth
mouse model
Hallmark
p.adj = 0.05
0
5
10
15
20
25
-log10(p.adj)

Table 2

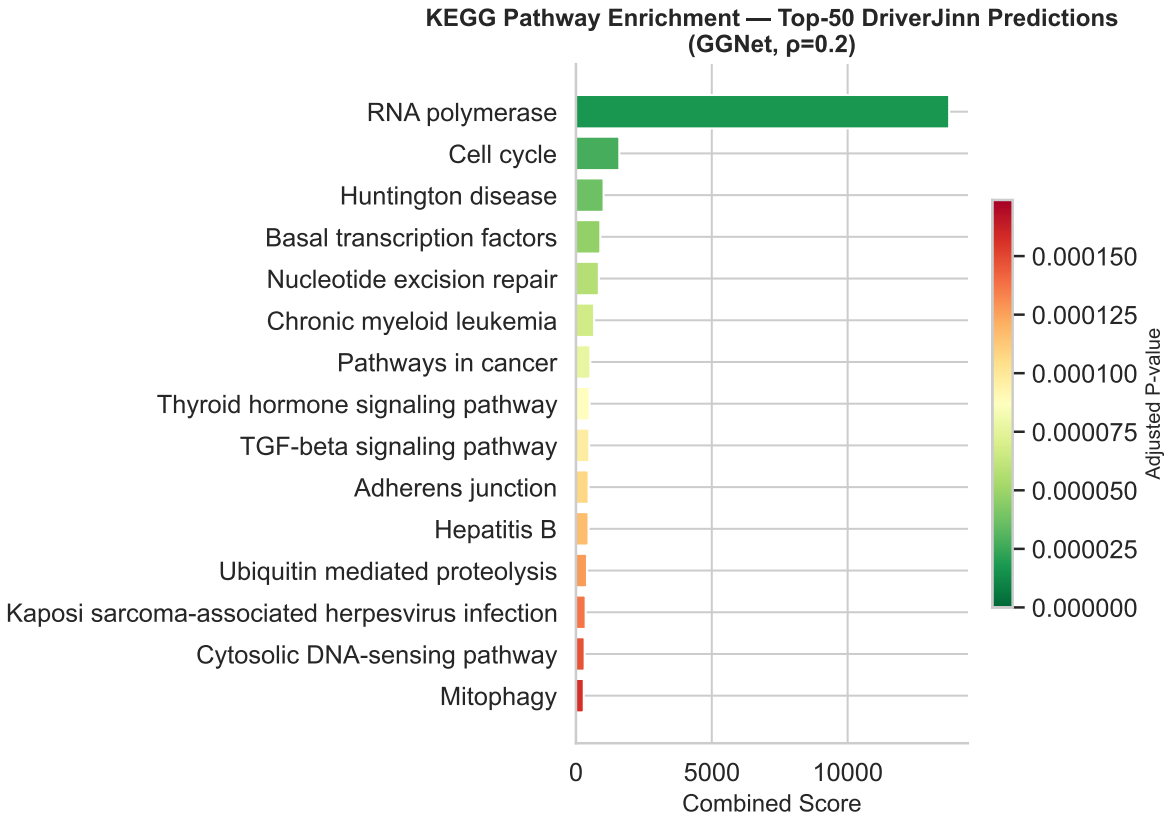
	Hallmark	Overlap	GS Size	p-value	p.adj
0	role in cancer	33	343	1.158e-25	1.401e-24
1	function summary	33	344	1.274e-25	1.401e-24
2	escaping programmed cell death	24	218	5.529e-20	4.055e-19
3	types of alteration in cancer	24	231	2.198e-19	1.209e-18
4	genome instability and mutations	18	119	1.211e-17	4.769e-17
5	invasion and metastasis	22	216	1.301e-17	4.769e-17
6	impact of mutation on function	20	189	2.170e-16	6.821e-16
7	differentiation and development	19	167	3.377e-16	9.285e-16
8	proliferative signalling	20	195	4.021e-16	9.830e-16
9	cell division control	17	129	1.148e-15	2.526e-15
10	tumour promoting inflammation	12	61	1.611e-13	3.223e-13
11	cell replicative immortality	10	51	1.949e-11	3.573e-11
12	escaping immune response to cancer	11	70	2.331e-11	3.682e-11
13	angiogenesis	12	91	2.343e-11	3.682e-11
14	senescence	10	63	1.785e-10	2.617e-10
15	fusion partner	12	148	7.134e-09	9.809e-09
16	clinical impact	11	119	7.924e-09	1.025e-08
17	global regulation of gene expression	9	72	1.355e-08	1.657e-08
18	change of cellular energetics	9	85	5.959e-08	6.900e-08
19	suppression of growth	9	132	2.598e-06	2.858e-06
20	mouse model	6	87	1.237e-04	1.296e-04

KEGG Pathway Enrichment

Requires `gseapy` (`pip install gseapy`) and internet access for Enrichr API.

```
try:
    import gseapy as gp
    top50_genes = genes.head(50)["gene_name"].tolist()
    enr = gp.enrichr(gene_list=top50_genes, gene_sets=["KEGG_2021_Human"],
                     organism="human", outdir=None, cutoff=0.05)
    results = (enr.results[enr.results["Adjusted P-value"] < 0.05]
               .sort_values("Combined Score", ascending=False).head(15))
    if results.empty:
        results = enr.results.sort_values("Adjusted P-value").head(15)
    fig, ax = plt.subplots(figsize=(8, 6))
```

```
    colors = plt.cm.RdYlGn_r(np.linspace(0.1, 0.9, len(results)))
    ax.barh(results["Term"].str.replace(r" \(.*\)", "", regex=True),
            results["Combined Score"], color=colors)
    sm = plt.cm.ScalarMappable(cmap="RdYlGn_r",
         norm=plt.Normalize(results["Adjusted P-value"].min(),
                            results["Adjusted P-value"].max()))
    sm.set_array([])
    plt.colorbar(sm, ax=ax, shrink=0.6).set_label("Adjusted P-value", fontsize=10)
    ax.set_xlabel("Combined Score", fontsize=12)
    ax.set_title("KEGG Pathway Enrichment - Top-50 DriverJinn Predictions\n(GGNet,  =0.2)",
                 fontsize=12, fontweight="bold")
    ax.invert_yaxis()
    sns.despine()
    plt.tight_layout()
    plt.savefig(PLOTS_DIR / "poster_kegg_enrichment.png", dpi=300, bbox_inches="tight")
    plt.show()
except ImportError:
    print("gseapy not installed - run: pip install gseapy")
```



Novel Candidate Genes

Top consensus-significant predictions not in any CGC tier:

```
if "consensus_significant" in ranked_genes.columns:
    novel_df = (
        ranked_genes
        .loc[ranked_genes["consensus_significant"] & ~ranked_genes["cgc_any"]]
        [["aggregate_rank", "gene_name", "mean_score", "folds_significant", "total_folds"]]
        .head(20)
    )
    display(novel_df.style.format({"mean_score": "{:.4f}"}).hide(axis="index"))
else:
    print("consensus_significant column not found - skipping novel candidate table")
```

Table 3

aggregate_rank	gene_name	mean_score	folds_significant	total_folds
42	MNAT1	1.0000	5	5
59	TRAF6	0.9999	5	5
69	MRE11	0.9999	5	5
84	GTF2H1	0.9999	5	5
96	PDS5B	0.9999	5	5
106	DYNC1H1	0.9999	5	5
124	AGO3	0.9999	5	5
133	GTF2H4	0.9999	5	5
147	RPTOR	0.9999	5	5
174	SYNE1	0.9999	5	5
186	ARRB2	0.9999	5	5
188	LRP2	0.9999	5	5
210	YWHAB	0.9999	5	5
211	VCL	0.9999	5	5
215	PAK1	0.9999	5	5
216	MARK3	0.9999	5	5
221	BTRC	0.9999	5	5
239	REV3L	0.9999	5	5
251	SNRPB	0.9999	5	5
257	VWF	0.9999	5	5

Cross-Model Consensus

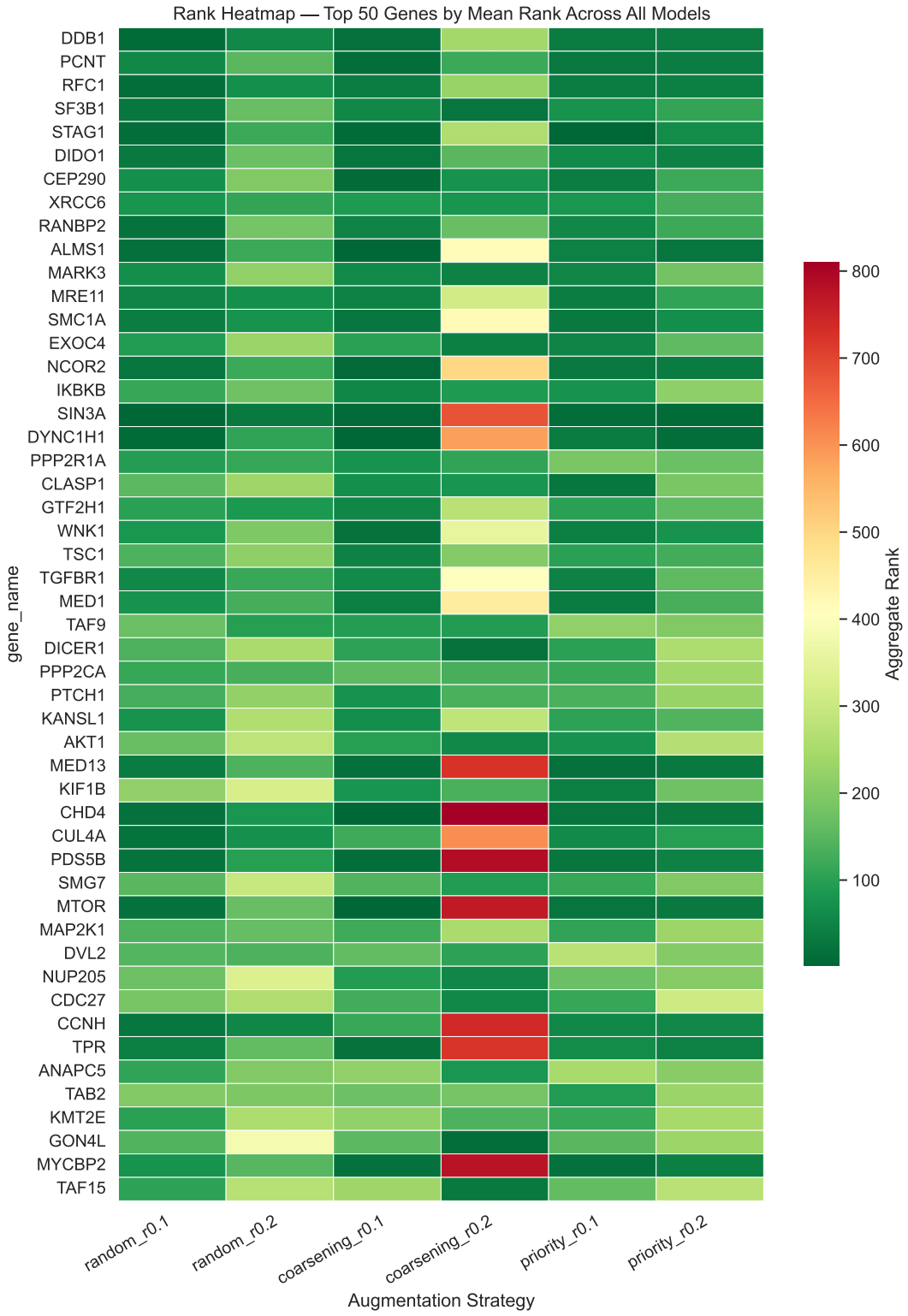
Combining ranked lists from 6 augmentation strategies to identify genes consistently predicted across strategies.

```
MODEL_LISTS = {
    "random_r0.1"    : "../model_results_2026-03-14/GGNet_random_r0.1_aggregated/GGNet_rando
    "random_r0.2"    : "../model_results_2026-03-14/GGNet_random_r0.2_aggregated/GGNet_rando
    "coarsening_r0.1": "../model_results_2026-03-15/GGNet_coarsening_r0.1_aggregated/GGNet_co
    "coarsening_r0.2": "../model_results_2026-03-15/GGNet_coarsening_r0.2_aggregated/GGNet_co
    "priority_r0.1"  : "../model_results_2026-03-15/GGNet_priority_r0.1_aggregated/GGNet_pri
    "priority_r0.2"  : "../model_results_2026-03-15/GGNet_priority_r0.2_aggregated/GGNet_pri
}

dfs = {}
for model, path in MODEL_LISTS.items():
    df = pd.read_csv(path)[["gene_name", "aggregate_rank", "mean_score", "consensus_significa
    df = df.rename(columns={"aggregate_rank": f"rank_{model}",
                            "mean_score": f"score_{model}",
                            "consensus_significant": f"consensus_{model}"})
    dfs[model] = df

merged_models = reduce(lambda l, r: pd.merge(l, r, on="gene_name", how="outer"), dfs.values()

rank_cols  = [f"rank_{m}"      for m in MODEL_LISTS]
score_cols = [f"score_{m}"     for m in MODEL_LISTS]
cons_cols  = [f"consensus_{m}" for m in MODEL_LISTS]

merged_models["mean_rank"]        = merged_models[rank_cols].mean(axis=1)
merged_models["mean_score_all"]   = merged_models[score_cols].mean(axis=1)
merged_models["models_consensus"] = merged_models[cons_cols].sum(axis=1).astype(int)
merged_models["cgc_tier"] = merged_models["gene_name"].map(
    lambda g: "Tier 1" if g in cgc_tier1 else ("Tier 2" if g in cgc_tier2 else "Novel")
)
cross_consensus = merged_models.sort_values("mean_rank").reset_index(drop=True)

print(f"Total genes across all models : {len(cross_consensus)}")
print(f"Consensus in  3/6 models      : {(cross_consensus['models_consensus'] >= 3).sum()}")
print(f"Consensus in  4/6 models      : {(cross_consensus['models_consensus'] >= 4).sum()}")
print(f"Consensus in all 6 models     : {(cross_consensus['models_consensus'] == 6).sum()}")
```

```
Total genes across all models : 11183
Consensus in  3/6 models      : 74
Consensus in  4/6 models      : 6
Consensus in all 6 models     : 0
```

Rank Heatmap — Top 50 by Mean Rank

```
top_n   = 50
top_ids = cross_consensus.head(top_n)["gene_name"].tolist()
heat_df = cross_consensus.set_index("gene_name").loc[top_ids, rank_cols].copy()
heat_df.columns = list(MODEL_LISTS.keys())
fig, ax = plt.subplots(figsize=(10, max(6, top_n * 0.28)))
sns.heatmap(heat_df, cmap="RdYlGn_r", ax=ax, linewidths=0.3, linecolor="white",
            cbar_kws={"label": "Aggregate Rank", "shrink": 0.6})
ax.set_xlabel("Augmentation Strategy")
ax.set_title(f"Rank Heatmap - Top {top_n} Genes by Mean Rank Across All Models")
ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha="right")
plt.tight_layout()
plt.show()
```


Rank Heatmap — Top 50 Genes by Mean Rank Across All Models
DDB1
PCNT
RFC1
SF3B1
STAG1
DIDO1
CEP290
XRCC6
RANBP2
ALMS1
MARK3
MRE11
SMC1A
EXOC4
NCOR2
IKBKB
SIN3A
DYNC1H1
PPP2R1A
CLASP1
GTF2H1
WNK1
TSC1
TGFBR1
MED1
TAF9
DICER1
PPP2CA
PTCH1
KANSL1
AKT1
MED13
KIF1B
CHD4
CUL4A
PDS5B
SMG7
MTOR
MAP2K1
DVL2
NUP205
CDC27
CCNH
TPR
ANAPC5
TAB2
KMT2E
GON4L
MYCBP2
TAF15
gene_name
random_r0.1
random_r0.2
coarsening_r0.1
coarsening_r0.2
priority_r0.1
priority_r0.2
Augmentation Strategy
800
700
600
500
400
300
200
100
Aggregate Rank

Top Cross-Model Consensus Genes (4/6 models)

```
cols_display = ["gene_name", "mean_rank", "mean_score_all", "models_consensus", "cgc_tier"] +
fmt = {"mean_rank": "{:.1f}", "mean_score_all": "{:.4f}"}
fmt.update({c: "{:.0f}" for c in rank_cols})
strong_4 = cross_consensus[cross_consensus["models_consensus"] >= 4][cols_display].reset_inde
strong_4.index += 1
strong_4.style.format(fmt).background_gradient(subset=rank_cols, cmap="RdYlGn_r", axis=None)
```

Table

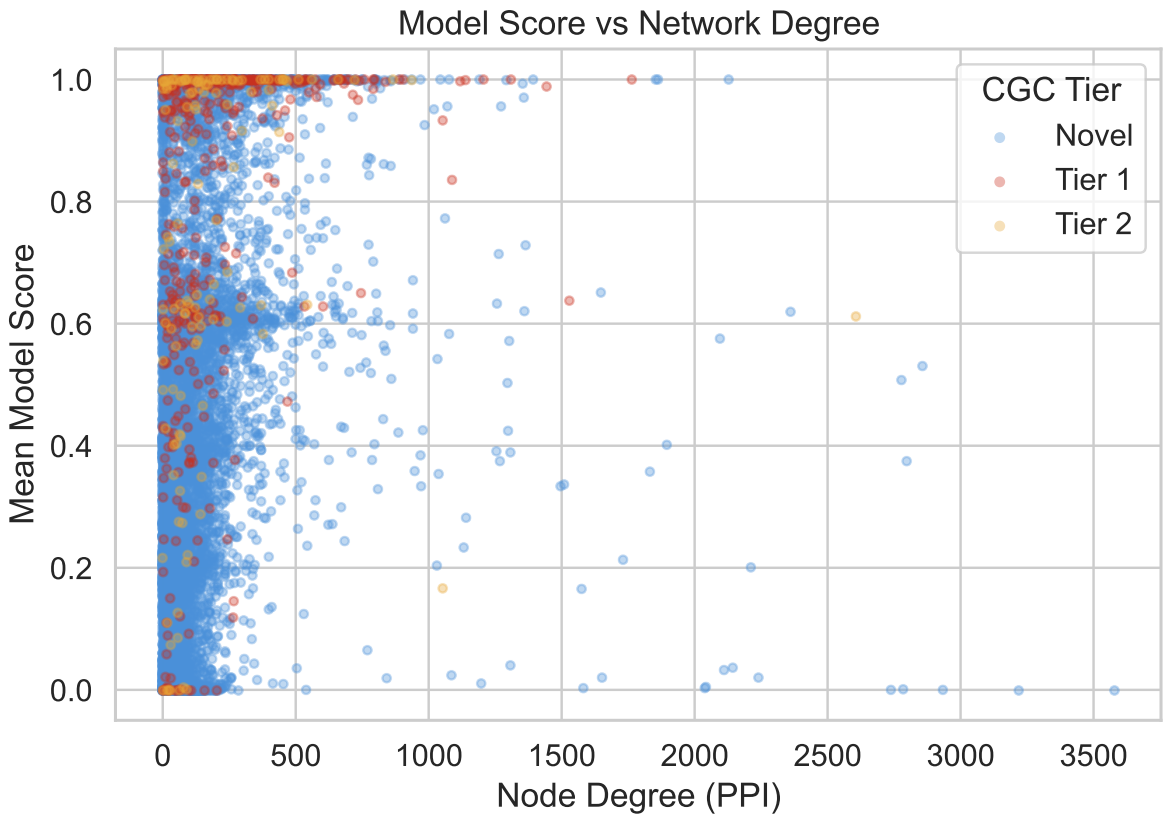
	gene_name	mean_rank	mean_score_all	models_consensus	cgc_tier	rank_random_r0.1	rank_r
1	TRERF1	439.5	0.6662	4	Novel	240	436
2	CBFA2T2	489.3	0.6862	4	Novel	441	693
3	ANAPC2	998.0	0.6053	4	Novel	1322	798
4	NPAS2	1112.3	0.5251	4	Novel	974	1190
5	SLC16A1	1419.7	0.4960	4	Novel	1569	1545
6	TUB	4395.7	0.2400	4	Novel	2837	4021

PPI Network Analysis

Degree Centrality vs Model Score

```
ranked_genes["degree"] = ranked_genes["gene_name"].map(
    lambda g: degree_map.get(name_to_idx.get(g), np.nan)
)

fig, ax = plt.subplots(figsize=(7, 5))
for tier, grp in ranked_genes.dropna(subset=["degree"]).groupby("cgc_label"):
    ax.scatter(grp["degree"], grp["mean_score"], alpha=0.35, s=12,
               color=tier_colors[tier], label=tier, rasterized=True)
ax.set_xlabel("Node Degree (PPI)")
ax.set_ylabel("Mean Model Score")
ax.set_title("Model Score vs Network Degree")
ax.legend(title="CGC Tier")
plt.tight_layout()
```

```
plt.show()

valid = ranked_genes.dropna(subset=["degree"])
rho, pval = spearmanr(valid["degree"], valid["mean_score"])
print(f"Spearman rho (degree vs score): {rho:.3f}  p={pval:.2e}")
```

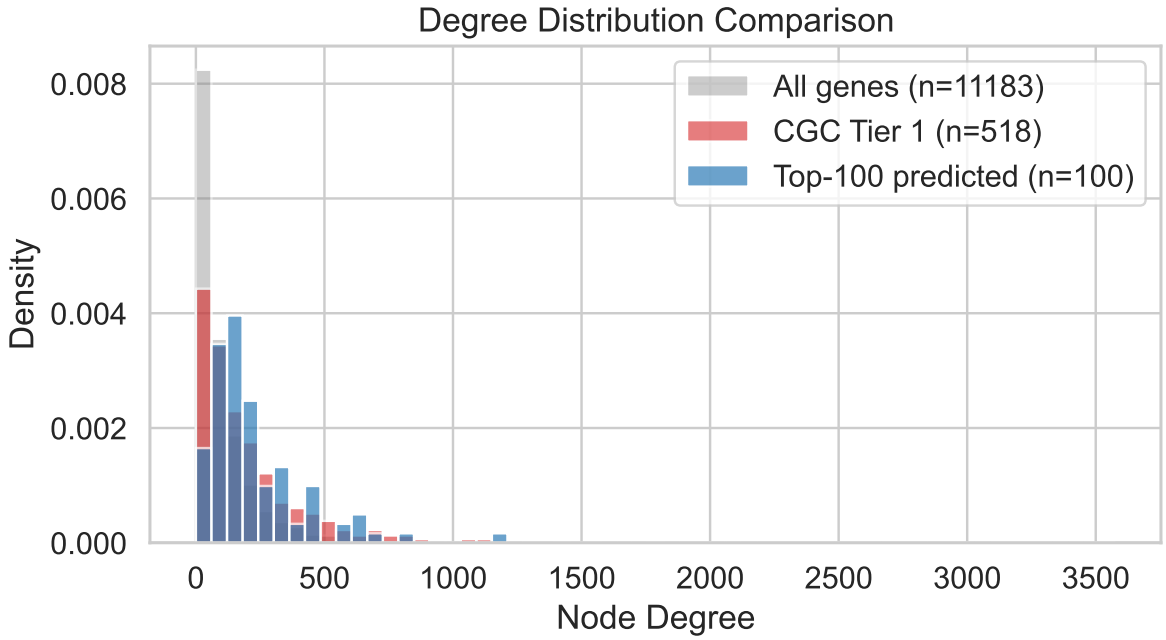



```
Spearman rho (degree vs score): 0.476  p=0.00e+00
```

Degree Distribution: Top-100 vs CGC vs All Genes

```
all_degrees    = [degree_map.get(name_to_idx.get(g), 0) for g in ranked_genes["gene_name"] i
top100_degrees = [degree_map.get(name_to_idx.get(g), 0) for g in ranked_genes.head(100)["gen
cgc1_degrees   = [degree_map.get(name_to_idx.get(g), 0) for g in ranked_genes[ranked_genes["

fig, ax = plt.subplots(figsize=(7, 4))
```

```
bins = np.linspace(0, max(all_degrees), 60)
ax.hist(all_degrees,    bins=bins, alpha=0.4, label=f"All genes (n={len(all_degrees)})",
ax.hist(cgc1_degrees,   bins=bins, alpha=0.6, label=f"CGC Tier 1 (n={len(cgc1_degrees)})",
ax.hist(top100_degrees, bins=bins, alpha=0.7, label=f"Top-100 predicted (n={len(top100_degre
ax.set_xlabel("Node Degree")
ax.set_ylabel("Density")
ax.set_title("Degree Distribution Comparison")
ax.legend()
plt.tight_layout()
plt.show()
print(f"Median degree - All: {np.median(all_degrees):.0f} | "
      f"CGC Tier 1: {np.median(cgc1_degrees):.0f} | "
      f"Top-100: {np.median(top100_degrees):.0f}")
```

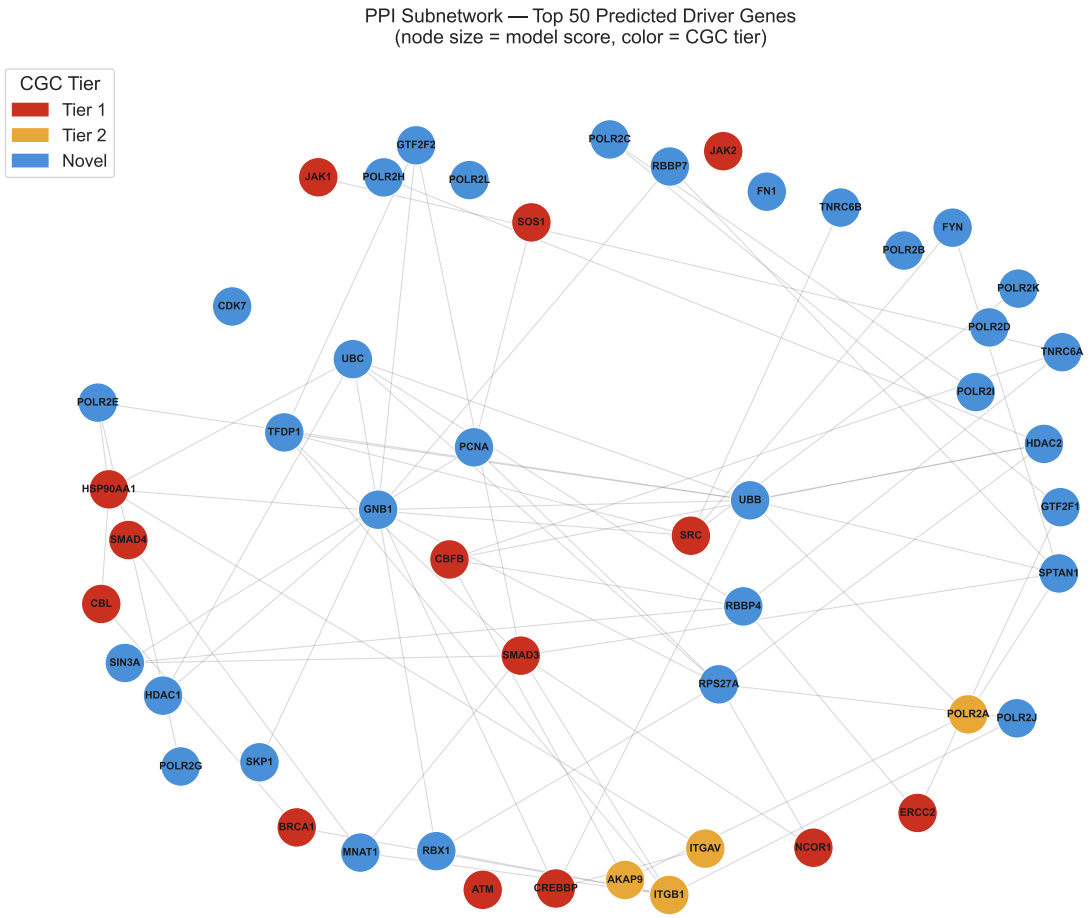



```
Median degree - All: 61 | CGC Tier 1: 126 | Top-100: 177
```

PPI Subnetwork of Top-50 Predicted Genes

```
top50_nodes = [name_to_idx[g] for g in genes.head(50)["gene_name"] if g in name_to_idx]
G_sub = G_full.subgraph(top50_nodes).copy()
```

```
G_named = nx.relabel_nodes(G_sub, id_to_name)

score_map_50  = genes.set_index("gene_name")["mean_score"].to_dict()
tier_map_50   = genes.set_index("gene_name")["cgc_label"].to_dict()
node_colors50 = [tier_colors.get(tier_map_50.get(n, "Novel"), "#1f77b4") for n in G_named.noc
node_sizes50  = [score_map_50.get(n, 0.5) * 800 for n in G_named.nodes()]

pos = nx.spring_layout(G_named, seed=42, k=1.5)
fig, ax = plt.subplots(figsize=(12, 10))
nx.draw_networkx_edges(G_named, pos, ax=ax, alpha=0.3, edge_color="grey", width=0.8)
nx.draw_networkx_nodes(G_named, pos, ax=ax, node_color=node_colors50,
                       node_size=node_sizes50, edgecolors="white", linewidths=0.8)
nx.draw_networkx_labels(G_named, pos, ax=ax, font_size=7.5, font_weight="bold")
ax.legend(handles=[Patch(color=c, label=t) for t, c in tier_colors.items()],
          title="CGC Tier", loc="upper left")
ax.set_title("PPI Subnetwork - Top 50 Predicted Driver Genes\n"
             "(node size = model score, color = CGC tier)", pad=12)
ax.axis("off")
plt.tight_layout()
plt.show()
print(f"Subgraph density: {nx.density(G_named):.4f}")
connected = list(nx.connected_components(G_named))
print(f"Connected components: {len(connected)} (largest: {max(len(c) for c in connected)} noc
```




```
Subgraph density: 0.0555
Connected components: 8 (largest: 43 nodes)
```

Cross-Network Consensus Table

Top 20 genes ranked consistently across GGNet, PathNet, and PPNet.

```
ggnet   = pd.read_csv("../model_results_2026-03-14/GGNet_random_r0.2_aggregated/"
                      "GGNet_random_r0.2_aggregated_all_genes.csv")
pathnet = pd.read_csv("../model_results_2026-03-20/PathNet_random_r0.2_aggregated/"
                      "PathNet_random_r0.2_aggregated_all_genes.csv")
```

```
ppnet   = pd.read_csv("../model_results_2026-03-20/PPNet_random_r0.2_aggregated/"
                      "PPNet_random_r0.2_aggregated_all_genes.csv")

def cgc_label(gene):
    if gene in cgc_tier1: return "Tier 1"
    if gene in cgc_tier2: return "Tier 2"
    return "Novel"

merged_net = (
    ggnet[["gene_name", "aggregate_rank", "mean_score"]]
    .rename(columns={"aggregate_rank": "GGNet_rank", "mean_score": "GGNet_score"})
    .merge(pathnet[["gene_name", "aggregate_rank", "mean_score"]]
           .rename(columns={"aggregate_rank": "PathNet_rank", "mean_score": "PathNet_score"})
           on="gene_name", how="inner")
    .merge(ppnet[["gene_name", "aggregate_rank", "mean_score"]]
           .rename(columns={"aggregate_rank": "PPNet_rank", "mean_score": "PPNet_score"}),
           on="gene_name", how="inner")
)
merged_net["CGC"] = merged_net["gene_name"].apply(cgc_label)
merged_net["consensus_rank"] = merged_net[["GGNet_rank", "PathNet_rank", "PPNet_rank"]].mean
merged_net = merged_net.sort_values("consensus_rank")

top20 = merged_net.head(20).copy()
top20.insert(0, "Consensus Rank", range(1, 21))
top20 = top20.rename(columns={"gene_name": "Gene", "GGNet_rank": "GGNet",
                              "PathNet_rank": "PathNet", "PPNet_rank": "PPNet",
                              "CGC": "CGC Status"})
display_cols = ["Consensus Rank", "Gene", "GGNet", "PathNet", "PPNet", "CGC Status"]

ggnet_scores = ggnet.set_index("gene_name")["mean_score"].to_dict()

def style_row(row):
    if row["CGC Status"] == "Novel":  return ["background-color: #FFF3CD; font-weight: bold"]
    if row["CGC Status"] == "Tier 1": return ["background-color: #FDECEA"] * len(row)
    return [""] * len(row)

styled = (
    top20[display_cols].style
    .apply(style_row, axis=1)
    .format({"GGNet": "{:.0f}", "PathNet": "{:.0f}", "PPNet": "{:.0f}"})
    .set_caption("Top 20 Consensus Cancer Driver Gene Predictions Across Networks "
                 "(DriverJinn,  =0.2) - Novel candidates highlighted in gold")
```

```
    .set_table_styles([
        {"selector": "caption", "props": [("font-size","13px"),("font-weight","bold"),
                                          ("text-align","left"),("padding-bottom","8px")]},
        {"selector": "th",      "props": [("background-color","#2C3E50"),("color","white"),
                                          ("font-size","11px"),("text-align","center"),
                                          ("padding","6px 10px")]},
        {"selector": "td",      "props": [("font-size","11px"),("text-align","center"),
                                          ("padding","5px 10px")]},
        {"selector": "tr:hover td", "props": [("background-color","#EBF5FB")]},
    ])
)

novel_top20 = top20[top20["CGC Status"] == "Novel"]["Gene"].tolist()
print(f"Novel candidates in top 20: {novel_top20}")
display(styled)

import dataframe_image as dffi
dffi.export(styled, str(PLOTS_DIR / "novel.png"), table_conversion="matplotlib", dpi=200)
```

```
Novel candidates in top 20: ['SIN3A', 'HDAC2', 'SMARCC1', 'KMT2E', 'SMARCC2', 'SMARCA2', 'RBI
```

Table 5: Top 20 Consensus Cancer Driver Gene Predictions Across Networks (DriverJinn, =0.2) — Novel candidates highlighted in gold

	Consensus Rank	Gene	GGNet	PathNet	PPNet	CGC Status
30	1	SIN3A	31	20	118	Novel
22	2	HDAC2	23	82	69	Novel
113	3	NCOR2	121	46	91	Tier 1
55	4	SMARCC1	58	149	85	Novel
224	5	KMT2E	255	22	23	Novel
17	6	SOS1	18	91	193	Tier 1
20	7	CBFB	21	84	209	Tier 1
93	8	KDR	100	113	103	Tier 1
85	9	SMARCC2	92	29	196	Novel
171	10	SMARCA2	190	68	61	Novel
12	11	RBBP4	13	47	294	Novel
196	12	EED	218	34	115	Tier 2
82	13	FRS2	88	132	147	Novel
254	14	DOCK7	290	37	64	Novel
280	15	NCOA3	319	4	80	Novel

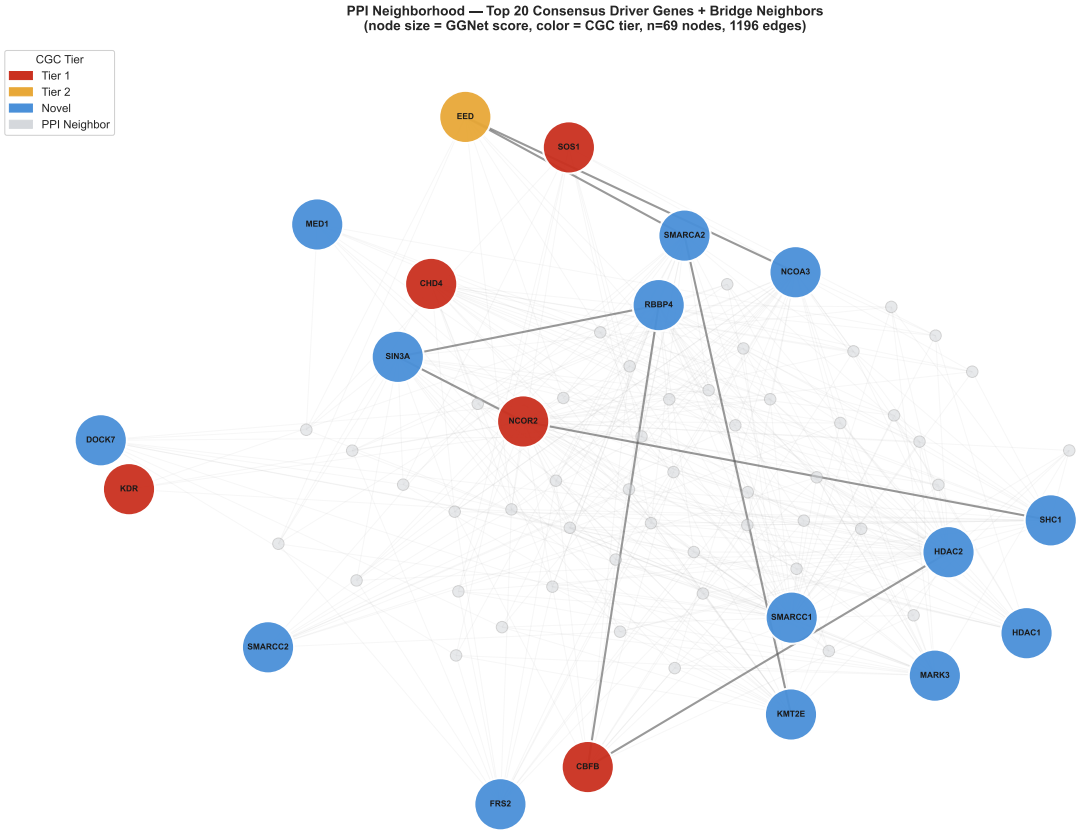
	Consensus Rank	Gene	GGNet	PathNet	PPNet	CGC Status
120	16	MED1	129	108	170	Novel
76	17	CHD4	82	19	315	Tier 1
95	18	SHC1	102	281	36	Novel
195	19	MARK3	216	107	100	Novel
2	20	HDAC1	3	122	301	Novel

PPI Neighborhood Subgraph (Consensus Top 20)

Top 20 consensus genes + bridge neighbors connecting 6 of them.

```
target_genes = top20["Gene"].tolist()
target_ids   = [name_to_id[g] for g in target_genes if g in name_to_id]
found_genes  = [g for g in target_genes if g in name_to_id]
target_id_set = set(target_ids)

# 1-hop neighbors connecting  6 target genes
neighbor_target_count = {}
for nid in target_ids:
    for nb in G_full.neighbors(nid):
        if nb not in target_id_set:
            neighbor_target_count[nb] = neighbor_target_count.get(nb, 0) + 1
filtered_neighbors = {nb for nb, cnt in neighbor_target_count.items() if cnt >= 6}
neighbor_ids = target_id_set | filtered_neighbors

H_raw = G_full.subgraph(neighbor_ids).copy()
H     = nx.relabel_nodes(H_raw, id_to_name)

target_set    = set(found_genes)
cgc_color_map = {"Tier 1": "#ca301f", "Tier 2": "#E8A838", "Novel": "#4A90D9"}
cgc_lookup    = top20.set_index("Gene")["CGC Status"].to_dict()

# Edges
target_edges  = [(u, v) for u, v in H.edges() if u in target_set and v in target_set]
spoke_edges   = [(u, v) for u, v in H.edges() if (u in target_set) != (v in target_set)]

fig, ax = plt.subplots(figsize=(16, 12))
pos = nx.spring_layout(H, seed=42, k=1.5)
```

```
nx.draw_networkx_edges(H, pos, edgelist=spoke_edges,  ax=ax, alpha=0.10, edge_color="#AAAAAA"
nx.draw_networkx_edges(H, pos, edgelist=target_edges, ax=ax, alpha=0.60, edge_color="#555555"

neighbor_nodes = [g for g in H.nodes() if g not in target_set]
nx.draw_networkx_nodes(H, pos, nodelist=neighbor_nodes, ax=ax,
                       node_color="#D5D8DC", node_size=120,
                       edgecolors="#AAAAAA", linewidths=0.8, alpha=0.55)
target_in_H = list(target_set & set(H.nodes()))
nx.draw_networkx_nodes(H, pos, nodelist=target_in_H, ax=ax,
                       node_color=[cgc_color_map.get(cgc_lookup.get(g, "Novel"), "#4A90D9")
                                   for g in target_in_H],
                       node_size=[max(ggnet_scores.get(g, 0.5), 0.1) * 2500 for g in target_i
                       edgecolors="white", linewidths=2.0, alpha=0.95)
nx.draw_networkx_labels(H, pos, labels={g: g for g in target_set if g in H.nodes()},
                        ax=ax, font_size=8, font_weight="bold", font_color="#1A1A1A")

patches = [mpatches.Patch(color=c, label=l) for l, c in cgc_color_map.items()]
patches.append(mpatches.Patch(color="#D5D8DC", label="PPI Neighbor"))
ax.legend(handles=patches, title="CGC Tier", fontsize=11, title_fontsize=11,
          loc="upper left", framealpha=0.9)
ax.set_title(f"PPI Neighborhood - Top 20 Consensus Driver Genes + Bridge Neighbors\n"
             f"(node size = GGNet score, color = CGC tier, "
             f"n={H.number_of_nodes()} nodes, {H.number_of_edges()} edges)",
             fontsize=13, fontweight="bold", pad=14)
ax.axis("off")
plt.tight_layout()
plt.savefig(PLOTS_DIR / "consensus_subnetwork.png", dpi=300, bbox_inches="tight")
plt.savefig(PLOTS_DIR / "consensus_subnetwork.png", dpi=200, bbox_inches="tight")
plt.show()
print(f"Target genes in graph : {sorted(target_set & set(H.nodes()))}")
print(f"Total nodes           : {H.number_of_nodes()}")
print(f"Total edges           : {H.number_of_edges()}")
```




```
Target genes in graph : ['CBFB', 'CHD4', 'DOCK7', 'EED', 'FRS2', 'HDAC1', 'HDAC2', 'KDR', 'KI
Total nodes           : 69
Total edges           : 1196
```

Chromatin Remodeling Cluster — Deep Dive

```
#   Cluster definition
CLUSTER_GENES = ["SIN3A", "HDAC1", "HDAC2", "SMARCC1", "SMARCC2", "RBBP4", "KMT2E"]

COMPLEX_MAP = {
    "SIN3A":  "SIN3/HDAC",
    "HDAC1":  "SIN3/HDAC",
```

```
    "HDAC2":  "SIN3/HDAC",
    "RBBP4":  "NuRD/PRC2",
    "SMARCC1": "SWI/SNF",
    "SMARCC2": "SWI/SNF",
    "KMT2E":  "KMT",
}

COMPLEX_COLORS = {
    "SIN3/HDAC": "#E74C3C",
    "NuRD/PRC2": "#9B59B6",
    "SWI/SNF":   "#27AE60",
    "KMT":       "#2980B9",
}

cluster_ids  = [name_to_id[g] for g in CLUSTER_GENES if g in name_to_id]
cluster_set  = set(CLUSTER_GENES)

print("Cluster genes found in graph:",
      [g for g in CLUSTER_GENES if g in name_to_id])
```

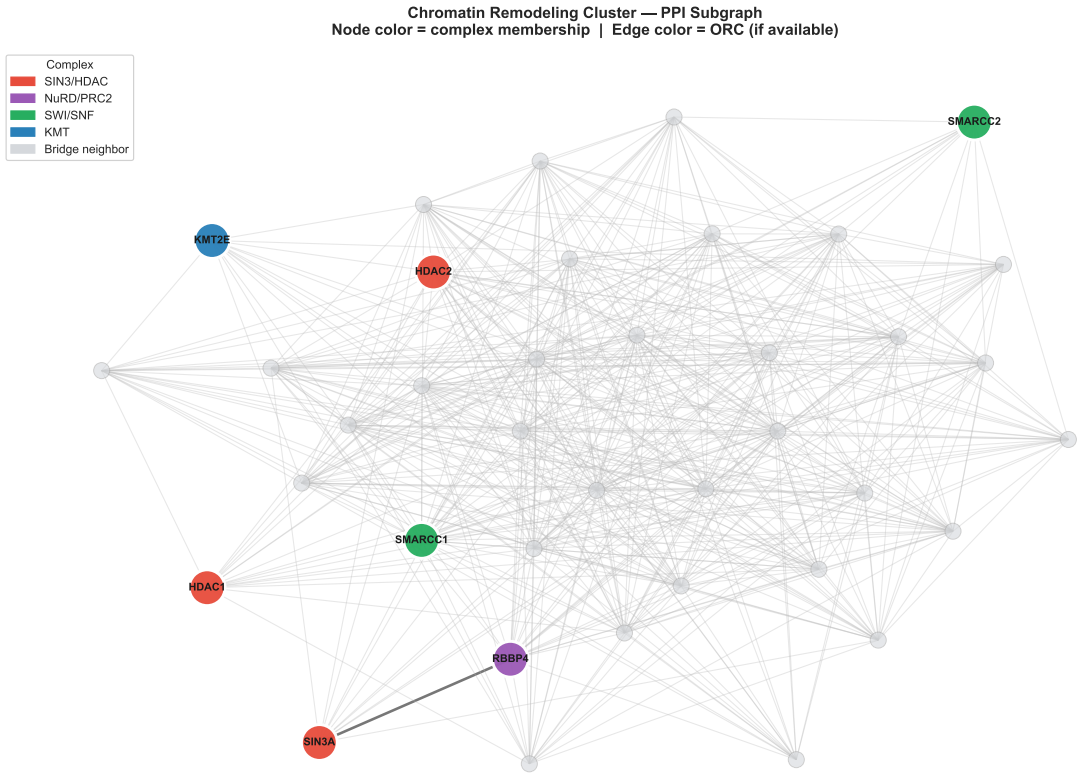
```
Cluster genes found in graph: ['SIN3A', 'HDAC1', 'HDAC2', 'SMARCC1', 'SMARCC2', 'RBBP4', 'KM
```

Analysis 1 — Curvature-Annotated Cluster Subgraph

```
#   1-hop neighbors shared between  4 cluster genes
nb_count = {}
for nid in cluster_ids:
    for nb in G_full.neighbors(nid):
        if nb not in set(cluster_ids):
            nb_count[nb] = nb_count.get(nb, 0) + 1
bridge_ids = {nb for nb, cnt in nb_count.items() if cnt >= 4}
subgraph_ids = set(cluster_ids) | bridge_ids

C_raw  = G_full.subgraph(subgraph_ids).copy()
C      = nx.relabel_nodes(C_raw, id_to_name)

#   Extract ORC values from edge attributes
orc_values = []
for u, v, data in C.edges(data=True):
    orc = data.get("ricciCurvature", data.get("OllivierRicci", np.nan))
```

```
    orc_values.append(orc)

has_orc = not all(np.isnan(v) if isinstance(v, float) else False for v in orc_values)

#   Layout
fig, ax = plt.subplots(figsize=(14, 10))
pos = nx.spring_layout(C, seed=7, k=2.0)

# Edge coloring by ORC (blue = negative/bottleneck, white = neutral, red = positive)
if has_orc:
    edge_list = list(C.edges())
    edge_orc  = [C[u][v].get("ricciCurvature",
                  C[u][v].get("OllivierRicci", 0)) for u, v in edge_list]
    vmin, vmax = min(edge_orc), max(edge_orc)
    norm    = plt.Normalize(vmin=vmin, vmax=vmax)
    cmap    = plt.cm.RdYlBu
    e_colors = [cmap(norm(v)) for v in edge_orc]

    # Cluster edges thicker
    e_widths = [2.5 if (u in cluster_set and v in cluster_set) else 1.0
                for u, v in edge_list]
    e_alphas = [0.85 if (u in cluster_set and v in cluster_set) else 0.40
                for u, v in edge_list]

    for (u, v), color, width, alpha in zip(edge_list, e_colors, e_widths, e_alphas):
        nx.draw_networkx_edges(C, pos, edgelist=[(u, v)], ax=ax,
                               edge_color=[color], width=width, alpha=alpha)

    sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
    sm.set_array([])
    plt.colorbar(sm, ax=ax, shrink=0.5, pad=0.01,
                 label="Ollivier-Ricci Curvature")
else:
    # Fallback: grey edges
    cluster_edges = [(u, v) for u, v in C.edges()
                     if u in cluster_set and v in cluster_set]
    bridge_edges  = [(u, v) for u, v in C.edges()
                     if u not in cluster_set or v not in cluster_set]
    nx.draw_networkx_edges(C, pos, edgelist=cluster_edges, ax=ax,
                           edge_color="#555555", width=2.2, alpha=0.8)
    nx.draw_networkx_edges(C, pos, edgelist=bridge_edges,  ax=ax,
                           edge_color="#BBBBBB", width=0.9, alpha=0.35)
```

```
#   Nodes
bridge_nodes  = [g for g in C.nodes() if g not in cluster_set]
cluster_nodes = [g for g in C.nodes() if g in cluster_set]

nx.draw_networkx_nodes(C, pos, nodelist=bridge_nodes, ax=ax,
                       node_color="#D5D8DC", node_size=180,
                       edgecolors="#AAAAAA", linewidths=0.8, alpha=0.6)
nx.draw_networkx_nodes(C, pos, nodelist=cluster_nodes, ax=ax,
                       node_color=[COMPLEX_COLORS[COMPLEX_MAP[g]] for g in cluster_nodes],
                       node_size=900, edgecolors="white", linewidths=2.5, alpha=0.95)
nx.draw_networkx_labels(C, pos,
                        labels={g: g for g in cluster_nodes},
                        ax=ax, font_size=9, font_weight="bold")

# Legend: complex membership
legend_patches = [mpatches.Patch(color=c, label=l)
                  for l, c in COMPLEX_COLORS.items()]
legend_patches.append(mpatches.Patch(color="#D5D8DC", label="Bridge neighbor"))
ax.legend(handles=legend_patches, title="Complex", fontsize=10,
          title_fontsize=10, loc="upper left", framealpha=0.9)

ax.set_title("Chromatin Remodeling Cluster - PPI Subgraph\n"
             "Node color = complex membership  |  Edge color = ORC (if available)",
             fontsize=13, fontweight="bold", pad=12)
ax.axis("off")
plt.tight_layout()
plt.savefig(PLOTS_DIR / "cluster_curvature_subgraph.png", dpi=300, bbox_inches="tight")
plt.savefig(PLOTS_DIR / "cluster_curvature_subgraph.png", dpi=200, bbox_inches="tight")
plt.show()
print(f"Cluster subgraph: {C.number_of_nodes()} nodes, {C.number_of_edges()} edges")
print(f"ORC available on edges: {has_orc}")
```



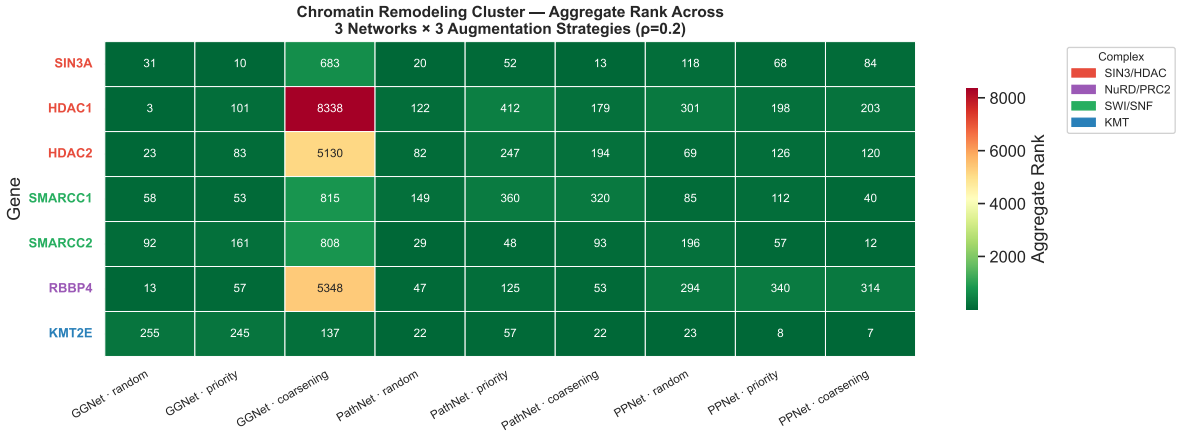

```
Cluster subgraph: 38 nodes, 497 edges
ORC available on edges: False
```

Analysis 2 — Cross-Network × Cross-Strategy Rank Heatmap

```
CONDITIONS = {
    # (network, strategy, r) → csv path
    "GGNet · random":      "../model_results_2026-03-14/GGNet_random_r0.2_aggregated/GGNet_ra
    "GGNet · priority":    "../model_results_2026-03-15/GGNet_priority_r0.2_aggregated/GGNet_
    "GGNet · coarsening":  "../model_results_2026-03-15/GGNet_coarsening_r0.2_aggregated/GGNe
    "PathNet · random":    "../model_results_2026-03-20/PathNet_random_r0.2_aggregated/PathNe
    "PathNet · priority":  "../model_results_2026-03-20/PathNet_priority_r0.2_aggregated/Path
    "PathNet · coarsening":"../model_results_2026-03-20/PathNet_coarsening_r0.2_aggregated/Pa
    "PPNet · random":      "../model_results_2026-03-20/PPNet_random_r0.2_aggregated/PPNet_ra
    "PPNet · priority":    "../model_results_2026-03-20/PPNet_priority_r0.2_aggregated/PPNet_
    "PPNet · coarsening":  "../model_results_2026-03-20/PPNet_coarsening_r0.2_aggregated/PPNe
```

```
}

rank_rows = {g: {} for g in CLUSTER_GENES}
for condition, path in CONDITIONS.items():
    try:
        df = pd.read_csv(path).set_index("gene_name")
        for g in CLUSTER_GENES:
            rank_rows[g][condition] = int(df.loc[g, "aggregate_rank"]) if g in df.index else
    except FileNotFoundError:
        for g in CLUSTER_GENES:
            rank_rows[g][condition] = np.nan

rank_df = pd.DataFrame(rank_rows).T   # genes × conditions
rank_df.index.name = "Gene"

# Single-axes heatmap - gene labels coloured by complex membership
fig, ax = plt.subplots(figsize=(13, 5))

sns.heatmap(rank_df, ax=ax, cmap="RdYlGn_r",
            annot=True, fmt=".0f", annot_kws={"size": 9},
            linewidths=0.4, linecolor="white",
            yticklabels=rank_df.index,
            cbar_kws={"label": "Aggregate Rank", "shrink": 0.7})

# Colour gene name y-tick labels by complex (replaces the strip subplot)
for tick, g in zip(ax.get_yticklabels(), rank_df.index):
    tick.set_color(COMPLEX_COLORS[COMPLEX_MAP[g]])
    tick.set_fontweight("bold")
    tick.set_fontsize(10)

ax.set_xlabel("")
ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha="right", fontsize=9)
ax.set_title("Chromatin Remodeling Cluster - Aggregate Rank Across\n"
             "3 Networks × 3 Augmentation Strategies ( =0.2)",
             fontsize=12, fontweight="bold")

# Legend outside the plot to the right, clear of the heatmap and colorbar
ax.legend(handles=[mpatches.Patch(color=c, label=l) for l, c in COMPLEX_COLORS.items()],
          title="Complex", fontsize=9, title_fontsize=9,
          loc="upper left", bbox_to_anchor=(1.18, 1), framealpha=0.9)

plt.tight_layout()
```

```
plt.savefig(PLOTS_DIR / "cluster_rank_heatmap.png", dpi=300, bbox_inches="tight")
plt.savefig(PLOTS_DIR / "cluster_rank_heatmap.png", dpi=200, bbox_inches="tight")
plt.show()

print("\nMean rank per gene across all conditions:")
print(rank_df.mean(axis=1).sort_values().round(1))
print(f"\nRank std (consistency - lower = more stable):")
print(rank_df.std(axis=1).sort_values().round(1))
```




```
Mean rank per gene across all conditions:
Gene
KMT2E        86.2
SIN3A       119.9
SMARCC2     166.2
SMARCC1     221.3
HDAC2       674.9
RBBP4       732.3
HDAC1      1095.2
dtype: float64

Rank std (consistency - lower = more stable):
Gene
KMT2E       101.1
SIN3A       214.2
SMARCC2     248.0
SMARCC1     251.3
HDAC2      1672.0
```

```
RBBP4      1735.6
HDAC1      2718.6
dtype: float64
```

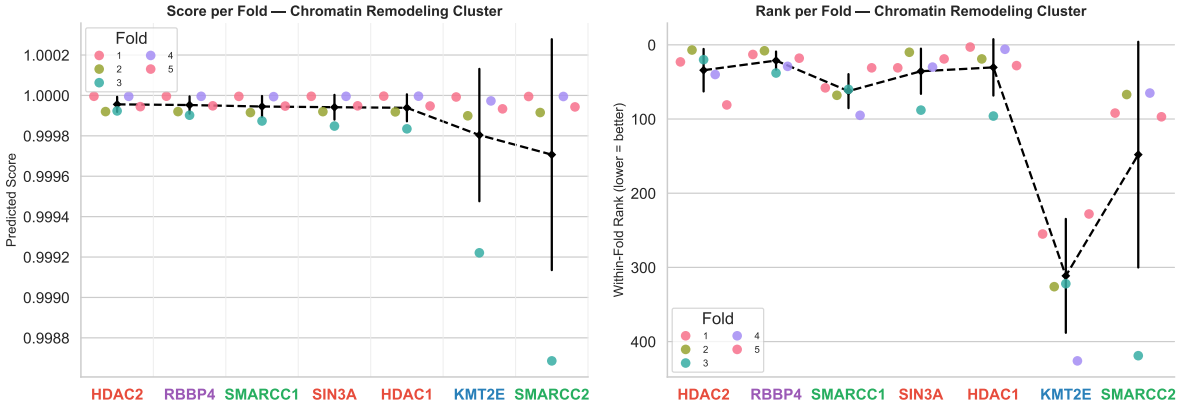
Analysis 3 — Fold-Level Score Consistency

```
fold_score_rows = []
for fold, df in fold_dfs.items():
    df_sorted = df.sort_values("rank").reset_index(drop=True)
    for g in CLUSTER_GENES:
        match = df_sorted[df_sorted["gene_name"] == g]
        fold_score_rows.append({
            "Gene":    g,
            "Fold":    fold,
            "Score":   float(match["driver_score"].iloc[0]) if not match.empty else np.nan,
            "Rank":    int(match["rank"].iloc[0])          if not match.empty else np.nan,
            "Complex": COMPLEX_MAP.get(g, "Unknown"),
        })

fold_scores = pd.DataFrame(fold_score_rows)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Panel A: score distribution per gene
gene_order = (fold_scores.groupby("Gene")["Score"].mean()
              .sort_values(ascending=False).index.tolist())

sns.stripplot(data=fold_scores, x="Gene", y="Score", hue="Fold",
              order=gene_order, palette="husl", dodge=True,
              size=8, alpha=0.85, ax=axes[0])
sns.pointplot(data=fold_scores, x="Gene", y="Score",
              order=gene_order, color="black",
              markers="D", linestyles="--", errorbar="sd",
              scale=0.7, ax=axes[0])
for i, g in enumerate(gene_order):
    axes[0].axvline(i + 0.5, color="#EEEEEE", linewidth=0.8)
    color = COMPLEX_COLORS[COMPLEX_MAP[g]]
    axes[0].get_xticklabels()  # ensure rendered
axes[0].set_xlabel("")
axes[0].set_ylabel("Predicted Score", fontsize=11)
axes[0].set_title("Score per Fold - Chromatin Remodeling Cluster", fontsize=12, fontweight="
```

```
axes[0].legend(title="Fold", fontsize=8, ncol=2)
# Colour x-tick labels by complex
for tick, g in zip(axes[0].get_xticklabels(), gene_order):
    tick.set_color(COMPLEX_COLORS[COMPLEX_MAP[g]])
    tick.set_fontweight("bold")
sns.despine(ax=axes[0])

# Panel B: rank per gene across folds
sns.stripplot(data=fold_scores, x="Gene", y="Rank", hue="Fold",
              order=gene_order, palette="husl", dodge=True,
              size=8, alpha=0.85, ax=axes[1])
sns.pointplot(data=fold_scores, x="Gene", y="Rank",
              order=gene_order, color="black",
              markers="D", linestyles="--", errorbar="sd",
              scale=0.7, ax=axes[1])
axes[1].invert_yaxis()
axes[1].set_xlabel("")
axes[1].set_ylabel("Within-Fold Rank (lower = better)", fontsize=11)
axes[1].set_title("Rank per Fold - Chromatin Remodeling Cluster", fontsize=12, fontweight="bo
axes[1].legend(title="Fold", fontsize=8, ncol=2)
for tick, g in zip(axes[1].get_xticklabels(), gene_order):
    tick.set_color(COMPLEX_COLORS[COMPLEX_MAP[g]])
    tick.set_fontweight("bold")
sns.despine(ax=axes[1])

plt.tight_layout()
plt.savefig(PLOTS_DIR / "cluster_fold_consistency.png", dpi=300, bbox_inches="tight")
plt.savefig(PLOTS_DIR / "cluster_fold_consistency.png", dpi=200, bbox_inches="tight")
plt.show()

print("\nScore CV (std/mean) per gene - lower = more consistent across folds:")
cv = fold_scores.groupby("Gene")["Score"].agg(["mean","std"])
cv["CV"] = (cv["std"] / cv["mean"]).round(3)
print(cv.sort_values("CV"))
```

```
C:\Users\siddh\AppData\Local\Temp\ipykernel_81720\3703365513.py:25: UserWarning:

The `scale` parameter is deprecated and will be removed in v0.15.0. You can now control the s

  sns.pointplot(data=fold_scores, x="Gene", y="Score",
C:\Users\siddh\AppData\Local\Temp\ipykernel_81720\3703365513.py:47: UserWarning:
```

```
The `scale` parameter is deprecated and will be removed in v0.15.0. You can now control the s

  sns.pointplot(data=fold_scores, x="Gene", y="Rank",
```

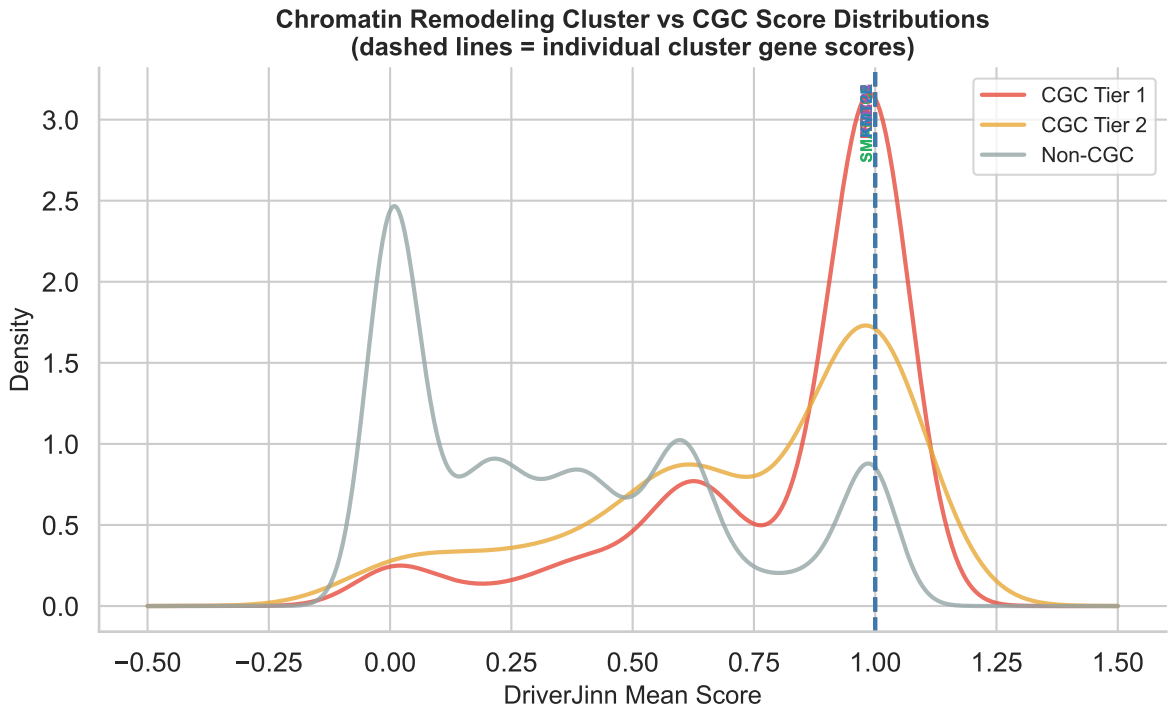



```
Score CV (std/mean) per gene - lower = more consistent across folds:
              mean       std     CV
Gene
HDAC1     0.999939  0.000067  0.000
HDAC2     0.999956  0.000038  0.000
KMT2E     0.999804  0.000328  0.000
RBBP4     0.999952  0.000043  0.000
SIN3A     0.999942  0.000061  0.000
SMARCC1   0.999945  0.000052  0.000
SMARCC2   0.999707  0.000572  0.001
```

Analysis 4 — Score vs Known Tier 1 Drivers

```python
fig, ax = plt.subplots(figsize=(8, 5))

# KDE for each CGC tier
for tier, color, label in [("T1", COLORS["T1"], "CGC Tier 1"),
                           ("T2", COLORS["T2"], "CGC Tier 2"),
                           ("non-CGC", COLORS["non-CGC"], "Non-CGC")]:
    subset = genes[genes["cgc_tier"] == tier]["mean_score"]
    subset.plot.kde(ax=ax, color=color, linewidth=2.0, label=label, alpha=0.8)
```

```
# Overlay cluster genes as vertical ticks
ymax = ax.get_ylim()[1] if ax.get_ylim()[1] > 0 else 5
for g in CLUSTER_GENES:
    score_row = genes[genes["gene_name"] == g]
    if score_row.empty:
        continue
    score = float(score_row["mean_score"].iloc[0])
    color = COMPLEX_COLORS[COMPLEX_MAP[g]]
    ax.axvline(score, color=color, linewidth=1.8, linestyle="--", alpha=0.85)
    ax.text(score, ax.get_ylim()[1] * 0.97, g,
            rotation=90, va="top", ha="right",
            fontsize=7.5, fontweight="bold", color=color)

ax.set_xlabel("DriverJinn Mean Score", fontsize=12)
ax.set_ylabel("Density", fontsize=12)
ax.set_title("Chromatin Remodeling Cluster vs CGC Score Distributions\n"
             "(dashed lines = individual cluster gene scores)",
             fontsize=12, fontweight="bold")
ax.legend(fontsize=10)
sns.despine()
plt.tight_layout()
plt.savefig(PLOTS_DIR / "cluster_vs_tier1_scores.png", dpi=300, bbox_inches="tight")
plt.savefig(PLOTS_DIR / "cluster_vs_tier1_scores.png", dpi=200, bbox_inches="tight")
plt.show()

# Print where each cluster gene falls relative to Tier 1 percentiles
t1_scores = genes[genes["cgc_tier"] == "T1"]["mean_score"]
print(f"\nCGC Tier 1 score range: {t1_scores.min():.4f} - {t1_scores.max():.4f}")
print(f"CGC Tier 1 median    : {t1_scores.median():.4f}\n")
for g in CLUSTER_GENES:
    row = genes[genes["gene_name"] == g]
    if row.empty: continue
    s = float(row["mean_score"].iloc[0])
    pct = (t1_scores < s).mean() * 100
    print(f"  {g:10s}  score={s:.4f}  above {pct:.0f}% of Tier 1 genes")
```




```
CGC Tier 1 score range: 0.0001 - 1.0000
CGC Tier 1 median    : 0.9867

  SIN3A       score=1.0000  above 98% of Tier 1 genes
  HDAC1       score=1.0000  above 100% of Tier 1 genes
  HDAC2       score=1.0000  above 98% of Tier 1 genes
  SMARCC1     score=0.9999  above 97% of Tier 1 genes
  SMARCC2     score=0.9999  above 94% of Tier 1 genes
  RBBP4       score=1.0000  above 99% of Tier 1 genes
  KMT2E       score=0.9999  above 85% of Tier 1 genes
```

Analysis 5 — Complex Membership Summary Table

```
KNOWN_CANCER_ROLE = {
    "SIN3A":  "Transcriptional co-repressor; recruits HDAC1/2. Tumour suppressor function in
    "HDAC1":  "Histone deacetylase; epigenetic silencing of tumour suppressors. Overexpressed
    "HDAC2":  "Histone deacetylase; promotes proliferation. Overexpressed in hepatocellular a
    "RBBP4":  "Core subunit of NuRD, PRC2, and CAF-1 complexes. PRC2 role links it to H3K27me
```

```
    "SMARCC1":"SWI/SNF ATPase subunit. SWI/SNF is mutated in ~20% of human cancers; acts as t
    "SMARCC2":"SWI/SNF core subunit. Co-mutated with SMARCC1 in tumours; contributes to chron
    "KMT2E":  "H3K4 mono-methyltransferase. Loss associated with AML; gains in solid tumours
}

CGCSTATUS = {g: cgc_label(g) for g in CLUSTER_GENES}

# Rebuild rank_df if Analysis 2 cell was not run
if "rank_df" not in globals():
    _rank_rows = {g: {} for g in CLUSTER_GENES}
    for condition, path in CONDITIONS.items():
        try:
            _df = pd.read_csv(path).set_index("gene_name")
            for g in CLUSTER_GENES:
                _rank_rows[g][condition] = int(_df.loc[g, "aggregate_rank"]) if g in _df.inde
        except FileNotFoundError:
            for g in CLUSTER_GENES:
                _rank_rows[g][condition] = np.nan
    rank_df = pd.DataFrame(_rank_rows).T

summary_data = []
for g in CLUSTER_GENES:
    row = genes[genes["gene_name"] == g]
    score  = float(row["mean_score"].iloc[0])   if not row.empty else np.nan
    agg_r  = int(row["aggregate_rank"].iloc[0]) if not row.empty else np.nan
    mean_r = rank_df.loc[g].mean() if g in rank_df.index else np.nan
    std_r  = rank_df.loc[g].std()  if g in rank_df.index else np.nan
    summary_data.append({
        "Gene":                g,
        "Complex":             COMPLEX_MAP[g],
        "CGC Status":          CGCSTATUS[g],
        "GGNet Score":         round(score, 4),
        "GGNet Rank":          agg_r,
        "Mean Rank (9 cond)": round(mean_r, 1),
        "Rank Std":            round(std_r, 1),
        "Known Cancer Role":   KNOWN_CANCER_ROLE[g],
    })

summary_df = pd.DataFrame(summary_data)

def style_cluster_row(row):
    color = COMPLEX_COLORS.get(row["Complex"], "#FFFFFF")
```

```
    # Lighten the complex colour for background
    return [f"background-color: {color}22"] * len(row)

styled_summary = (
    summary_df.style
    .apply(style_cluster_row, axis=1)
    .set_caption("Chromatin Remodeling Cluster - Gene Summary "
                 "(DriverJinn predictions,  =0.2, GGNet)")
    .set_table_styles([
        {"selector": "caption", "props": [("font-size","13px"),("font-weight","bold"),
                                          ("text-align","left"),("padding-bottom","8px")]},
        {"selector": "th",      "props": [("background-color","#2C3E50"),("color","white"),
                                          ("font-size","10px"),("text-align","center"),
                                          ("padding","5px 8px")]},
        {"selector": "td",      "props": [("font-size","10px"),("padding","5px 8px")]},
    ])
    .hide(axis="index")
)
display(styled_summary)

import dataframe_image as dffi
dffi.export(styled_summary, str(PLOTS_DIR / "cluster_summary_table.png"),
            table_conversion="matplotlib", dpi=200)
```

Table 6: Chromatin Remodeling Cluster
GGNet)

Gene	Complex	CGC Status	GGNet Score	GGNet Rank	Mean Rank (9 cond)	Rank Std
SIN3A	SIN3/HDAC	Novel	1.000000	31	119.900000	214.200000
HDAC1	SIN3/HDAC	Novel	1.000000	3	1095.200000	2718.600000
HDAC2	SIN3/HDAC	Novel	1.000000	23	674.900000	1672.000000
SMARCC1	SWI/SNF	Novel	0.999900	58	221.300000	251.300000
SMARCC2	SWI/SNF	Novel	0.999900	92	166.200000	248.000000
RBBP4	NuRD/PRC2	Novel	1.000000	13	732.300000	1735.600000
KMT2E	KMT	Novel	0.999900	255	86.200000	101.100000

```
ERROR   PropertyValue: Missing token for production Choice(ColorValue, Dimension, URIValue, V
ERROR   No content to parse.
ERROR   PropertyValue: Unknown syntax or no value:  #E74C3C22
ERROR   CSSStyleDeclaration: Syntax Error in Property: background-color: #E74C3C22
ERROR   PropertyValue: Missing token for production Choice(ColorValue, Dimension, URIValue, V
```

```
ERROR   No content to parse.
ERROR   PropertyValue: Unknown syntax or no value:  #27AE6022
ERROR   CSSStyleDeclaration: Syntax Error in Property: background-color: #27AE6022
ERROR   PropertyValue: Missing token for production Choice(ColorValue, Dimension, URIValue, V
ERROR   No content to parse.
ERROR   PropertyValue: Unknown syntax or no value:  #9B59B622
ERROR   CSSStyleDeclaration: Syntax Error in Property: background-color: #9B59B622
ERROR   PropertyValue: Missing token for production Choice(ColorValue, Dimension, URIValue, V
ERROR   No content to parse.
ERROR   PropertyValue: Unknown syntax or no value:  #2980B922
ERROR   CSSStyleDeclaration: Syntax Error in Property: background-color: #2980B922
```